University of Hertfordshire UH

School of Physics, Engineering and Computer Science

MSc Data Science Project

7PAM2002s

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Predicting Adult Obesity Prevalence Across English Local Authorities

Using Behavioural and Socioeconomic Machine Learning Indicators

Student Name and SRN:

Gabriel Lucky Lotanna — 24070357

Supervisor: Dhairya Kataria

Date Submitted: 22 April 2026

Word Count: Approximately 5000 words

GitHub: https://github.com/gabriel2lucky1990/adult-obesity-risk-modelling-england

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in Data Science at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at Assessment Offences and Academic Misconduct and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Gabriel Lucky Lotanna

Student Name signature: _____

Student SRN number: 24070357

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dhairya Kataria, for his guidance, constructive feedback, and consistent support throughout the development of this project. His insights into machine learning methodology and public health application significantly shaped the direction and quality of this work.

I am grateful to the University of Hertfordshire and the Department of Physics, Astronomy and Mathematics for providing the academic environment and resources that made this research possible.

I would also like to acknowledge NHS England and the Public Health Outcomes Framework team for making the NHS Fingertips dataset publicly available, without which this study could not have been conducted.

Finally, I extend my appreciation to my family and colleagues for their encouragement and patience throughout the duration of this programme.

Abstract

Adult obesity in England exhibits pronounced geographic variation, with Local Authority-level prevalence ranging from below 15% in the least deprived areas to above 40% in the most deprived — a disparity that conventional linear modelling approaches have struggled to adequately explain. This study applies four supervised machine learning models — Lasso Regression, Ridge Regression, Random Forest, and Gradient Boosting — to predict adult obesity prevalence across 147 English Local Authorities using five behavioural and socioeconomic indicators drawn from NHS Fingertips: physical inactivity, smoking prevalence, dietary quality (5-a-day compliance), Index of Multiple Deprivation (IMD) score, and the proportion of children in low-income households.

A hierarchical model selection framework incorporating cross-validation performance, residual diagnostic validity (Shapiro-Wilk normality test), and training-to-test generalisation stability identified Random Forest as the optimal model, achieving a cross-validated R^2 of 0.4388 and a test R^2 of 0.6012 with a mean absolute error of 3.22 percentage points — a 57% improvement in cross-validated generalisation over linear baselines. Gradient Boosting, despite its higher cross-validated R^2 (0.5080), was rejected due to significant overfitting (training-to-test residual ratio 10.7×) and failed normality diagnostics (Shapiro-Wilk $p = 0.0431$).

SHAP analysis identified IMD score (mean |SHAP| = 1.2815) and physical inactivity (1.2767) as near-equal primary predictors, with behavioural factors accounting for 58% of total predictive influence and socioeconomic factors 42%. The model correctly predicted 78.6% of test Local Authorities within ±5 percentage points, demonstrating clinically meaningful accuracy for public health resource allocation.

Keywords: adult obesity, machine learning, Random Forest, SHAP, NHS Fingertips, Local Authorities, socioeconomic deprivation.

Table of Contents

Declaration Statement....2
Acknowledgements....3
Abstract....4
List of Figures....7
List of Tables....8

Chapter 1: Introduction....9
1.1 Research Questions....9
1.2 Report Structure....10

Chapter 2: Literature Review....11
2.1 The Epidemiology and Determinants of Adult Obesity....11
2.2 Socioeconomic Deprivation and Health Inequalities....12
2.3 Machine Learning for Health Outcome Prediction....12
2.4 Model Explainability and SHAP Analysis....13
2.5 Research Gap and Study Contribution....13

Chapter 3: Data and Methodology....14
3.1 Data Sources and Variables....14
3.2 Exploratory Data Analysis....15
3.3 Ethical Considerations....16
3.4 Data Preparation....16
3.5 Hyperparameter Optimisation....17
3.6 Model Selection Framework....17
3.7 SHAP Explainability Analysis....18

Chapter 4: Results....18
4.1 Individual Model Performance....18
4.1.1 Lasso Regression....18
4.1.2 Ridge Regression....19
4.1.3 Random Forest....20
4.1.4 Gradient Boosting....21
4.2 Comprehensive Model Comparison....22

4.3 Model Selection Decision (RQ2) 23
4.4 Feature Importance Analysis (RQ1) 24
4.5 SHAP Analysis (RQ3) 25
4.6 Residual Diagnostics of Final Model 26
4.7 Prediction Accuracy Across Local Authorities 27

Chapter 5: Discussion of Results 28
5.1 Why Linear Models Were Insufficient 28
5.2 Why Random Forest Was Selected Over Gradient Boosting 28
5.3 Contextualising Performance Against Published Benchmarks 29
5.4 Interpretation of SHAP Findings 30
5.5 Limitations of the Study 30
5.6 Future Research Directions 31

Chapter 6: Conclusion 32
6.1 Policy Recommendations 33
6.2 Concluding Remarks 34

References 35
Appendix A: Complete Python Code 38

List of Figures

Figure 1: Distribution of Health and Socioeconomic Indicators across English Local Authorities....14

Figure 2: Pairwise Relationships Between All Variables (Diagonal = individual distributions)....14

Figure 3: Correlation Matrix for All Variables (r = 0.958 between IMD and child poverty confirmed)....15

Figure 4: Lasso Regression Model Diagnostics (R^2 = 0.4065) — Residuals, Q-Q, Actual vs Predicted....18

Figure 5: Ridge Regression Model Diagnostics (R^2 = 0.4186) — Note negative coefficient for child poverty....19

Figure 6: Random Forest Model Diagnostics (R^2 = 0.6012) — Selected final model....20

Figure 7: Gradient Boosting Model Diagnostics (R^2 = 0.6303) — Rejected due to overfitting....21

Figure 7a: Gradient Boosting Full Residual Diagnostic Analysis (Shapiro-Wilk p = 0.0431, FAILED)....21

Figure 8: Test R^2 and RMSE Comparison across all four models....22

Figure 9: Cross-Validation R^2 with error bars and Generalisation Gap across models....22

Figure 9a: Q-Q Plot Comparison — Random Forest vs Gradient Boosting normality of residuals....24

Figure 10: Random Forest Feature Importance for Predicting Adult Obesity Prevalence....24

Figure 11: SHAP Summary Beeswarm Plot — direction and magnitude of each feature's contribution....25

Figure 12: SHAP Feature Importance — Mean Absolute SHAP Values across all five predictors....26

Figure 13: SHAP Dependence Plots for IMD Score (left) and Physical Inactivity (right)....26

Figure 14: Residual Analysis — Random Forest Final Model (four-panel diagnostic plot)....27

Figure 15: Geographic Distribution of Obesity Prevalence Predictions across 117 test Local Authorities....27

List of Tables

Table 1: Descriptive Statistics for All Variables (n = 583 observations, 147 Local Authorities).....14

Table 2: GridSearchCV Hyperparameter Tuning Results — Parameter Grids and Optimal Values.....16

Table 3: Comprehensive Model Performance Comparison — R^2, Adj R^2, RMSE, MAE, CV R^2, Diagnostics..... 22

Chapter 1: Introduction

Adult obesity represents one of the most pressing public health challenges in England, with Local Authority-level prevalence ranging from below 15% to above 40% — a near threefold geographic variation reflecting complex interactions between behavioural and socioeconomic determinants (NHS Digital, 2023). The NHS spends an estimated £6.5 billion annually managing obesity-related conditions (Public Health England, 2023), yet conventional linear regression models struggle to capture the non-linear relationships and correlated predictors that drive this geographic heterogeneity, limiting the precision with which resources can be targeted (Friedman, 2001).

Machine learning offers a data-driven alternative capable of modelling these complex relationships without requiring prior specification of functional form (Breiman, 2001). Ensemble methods such as Random Forest and Gradient Boosting consistently outperform linear baselines in health prediction tasks (Alawad et al., 2019; Nguyen et al., 2021), while SHAP (SHapley Additive exPlanations) values provide transparent, theoretically grounded attribution of feature contributions, enabling policymakers to identify which determinants drive predictions in which areas (Lundberg and Lee, 2017).

This study applies four supervised machine learning models — Lasso Regression, Ridge Regression, Random Forest, and Gradient Boosting — to predict adult obesity prevalence across 147 English Local Authorities using five indicators from NHS Fingertips: physical inactivity, smoking prevalence, dietary quality (5-a-day compliance), Index of Multiple Deprivation (IMD) score, and children in low-income households. A hierarchical model selection framework incorporating cross-validation performance, residual diagnostics, and generalisation metrics identifies the optimal model, with SHAP analysis quantifying the relative contributions of behavioural versus socioeconomic determinants.

1.1 Research Questions

This study addresses three research questions:

- RQ1: Which behavioural and socioeconomic indicators are most strongly associated with adult obesity prevalence across English Local Authorities?
- RQ2: Which supervised machine learning model provides optimal predictive performance for area-level obesity modelling?
- RQ3: Do model explainability techniques reveal meaningful differences in the relative contributions of behavioural versus socioeconomic determinants?

1.2 Report Structure

Chapter 2 reviews the literature on obesity determinants, machine learning in health prediction, and SHAP explainability. Chapter 3 describes the data, preprocessing, and modelling framework. Chapter 4 presents model results including SHAP analysis. Chapter 5 critically discusses findings and their policy implications. Chapter 6 draws conclusions and policy recommendations.

Chapter 2: Literature Review

2.1 The Epidemiology and Determinants of Adult Obesity

Obesity is a multifactorial condition shaped by both individual behaviours and the structural environments in which people live. The World Health Organisation reports that global obesity prevalence has nearly tripled since 1975, with over 650 million adults classified as obese in 2016 (WHO, 2021). In England, Local Authorities in the most deprived decile consistently exhibit obesity rates 10–15 percentage points higher than their least deprived counterparts — a disparity that persists after controlling for demographic composition (Lyons et al., 2020).

Physical inactivity is the primary modifiable behavioural determinant of population-level obesity. Lee et al. (2012), in a landmark meta-analysis of 1.2 million participants, estimated that inactivity accounts for 6–10% of the global obesity burden. Dietary quality operates as an equally critical driver: Drewnowski and Specter (2004) demonstrated that energy-dense, nutrient-poor diets are disproportionately prevalent in deprived communities, reflecting economic constraints rather than personal preference alone — implying that behavioural programmes absent structural economic support are likely to be insufficient. Smoking, whilst associated with lower individual body weight, functions at the area level as a socioeconomic proxy: regions with high smoking rates share high deprivation, poor diet, and physical inactivity, creating analytical collinearity that challenges conventional regression (Jarvis and Wardle, 2006).

2.2 Socioeconomic Deprivation and Health Inequalities

The Marmot Review (2010) established that health outcomes — including obesity — follow a social gradient, with the most disadvantaged communities suffering disproportionately worse outcomes across virtually every indicator. Lyons et al. (2020) applied this framework to Local Authority obesity, finding IMD score to be the strongest single predictor, with each unit increase associated with a 0.31 percentage point rise in prevalence. Child poverty compounds this: children raised in poverty are more likely to develop obesogenic behaviours persisting into

adulthood, creating self-reinforcing cycles of disadvantage (Marmot, 2010). The near-perfect correlation between IMD and child poverty (r = 0.90+) produces multicollinearity that conventional linear models cannot reliably resolve — one of the strongest methodological justifications for machine learning in this context.

2.3 Machine Learning for Health Outcome Prediction

Obermeyer and Emanuel (2016) argued that machine learning represents a paradigm shift in health prediction, enabling models to learn complex patterns without pre-specified functional relationships. Subsequent evidence has borne this out: Weng et al. (2017) demonstrated that Random Forest outperformed conventional cardiovascular risk scores by 7.6% in a UK primary care dataset of 378,256 patients, while Alawad et al. (2019) reported AUC = 0.82 for Random Forest versus 0.71 for logistic regression in obesity risk classification. For area-level prediction, Nguyen et al. (2021) applied gradient boosting to US county-level obesity, reporting cross-validated R^2 = 0.61 — substantially above the 0.35–0.40 typical of linear models. Breiman (2001) provided the theoretical grounding for Random Forest's robustness: bootstrap aggregation across decorrelated trees reduces variance without increasing bias — a property especially valuable given the severe multicollinearity between IMD score and child poverty (r = 0.958) in the present dataset, where linear models produce unstable and sign-reversed coefficients. Lasso (Tibshirani, 1996) and Ridge (Hoerl and Kennard, 1970) regression provide interpretable linear baselines: Lasso's automatic variable selection and Ridge's coefficient shrinkage are directly relevant to this study's challenge: with five correlated predictors including two near-collinear deprivation measures (r = 0.958), these regularisation techniques serve as controlled baselines against which ensemble methods can be empirically compared.

2.4 Model Explainability and SHAP Analysis

A longstanding criticism of ensemble methods is their opacity. Lundberg and Lee (2017) addressed this through SHAP values, grounded in cooperative game theory, which assign each feature a contribution equal to its average marginal contribution

across all possible feature orderings. The resulting attributions are theoretically consistent, locally accurate, and — critically — handle correlated predictors correctly by measuring marginal rather than joint contributions. This makes SHAP particularly appropriate for the present dataset where IMD and child poverty are near-perfectly correlated: with $r^2 = 0.917$ ($r = 0.958$), joint attribution through split-based feature importance is unreliable, as it cannot isolate the marginal contribution of each deprivation measure independently. Lundberg et al. (2020) validated SHAP's clinical utility in hospital outcome prediction, while Chen et al. (2020) advocated for it as the preferred explainability method in public health contexts where both predictive accuracy and policy attribution are required.

2.5 Research Gap and Study Contribution

Despite growing evidence for machine learning in health prediction, its application to area-level obesity in the English Local Authority context remains limited. Existing studies focus on individual-level classification (Alawad et al., 2019) or US county-level data (Nguyen et al., 2021), leaving English NHS administrative data largely unaddressed. Few studies apply principled hierarchical model selection incorporating both predictive and diagnostic criteria — risking overfitted models that generalise poorly to new geographies. This study addresses both gaps, contributing a validated, explainable predictive model grounded in NHS Fingertips data that provides a replicable methodology for targeting public health resources across English Local Authorities.

Chapter 3: Data and Methodology

3.1 Data Sources and Variables

Data were obtained from NHS Fingertips (Public Health England, 2023), the national repository for local authority health indicators, covering 2020–2023. The final dataset comprised 583 observations across 147 English Local Authorities. Table 1 presents descriptive statistics for all variables.

Table 1: Descriptive Statistics for All Variables (n = 583, 147 Local Authorities)

Variable	Min	Max	Mean	Std Dev	Type
obesity_prevalence (%)	10.24	40.22	26.53	5.84	Target
physical_inactivity (%)	10.81	38.37	23.80	5.46	Behavioural
smoking_prevalence (%)	4.10	21.80	12.60	3.49	Behavioural
diet_5_a_day_pct (%)	18.84	49.52	34.20	5.85	Behavioural
imd_score	5.85	45.04	21.30	9.17	Socioeconomic
children_low_income_pct (%)	5.63	32.70	17.80	5.68	Socioeconomic

Source: NHS Fingertips, Public Health England (2023).

3.2 Exploratory Data Analysis

Figure 1 presents distribution histograms for all six variables. All variables display approximately normal distributions with no extreme outliers, supporting parametric modelling. Obesity prevalence ranges from 10.24% to 40.22% (mean = 26.53%, SD = 5.84%), reflecting substantial geographic variation across England.

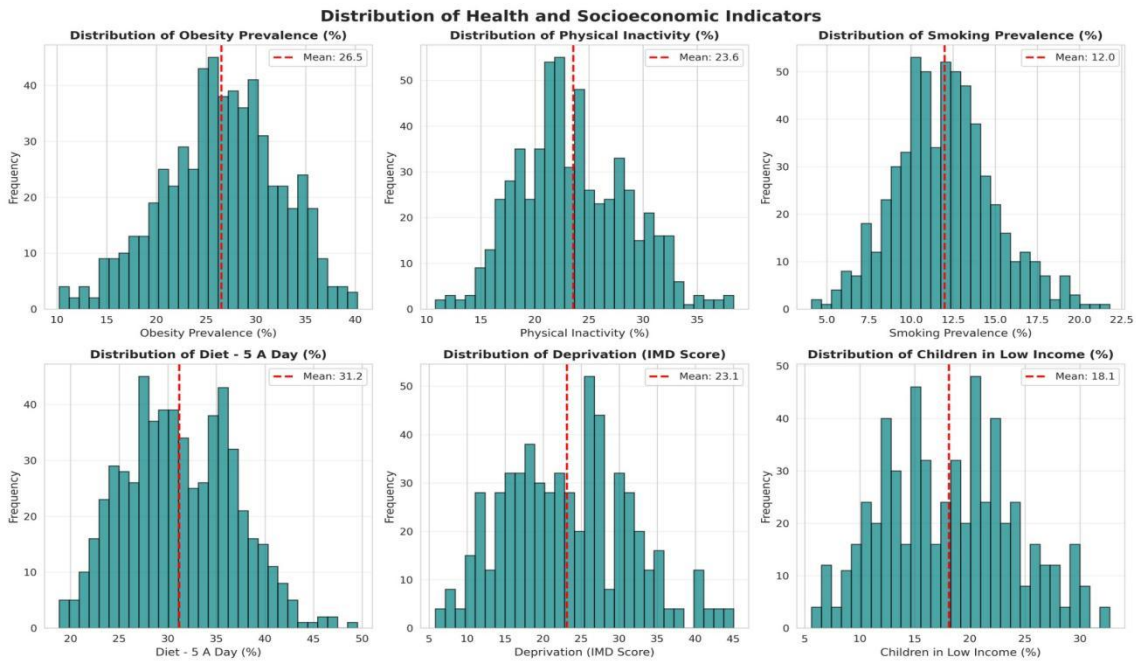

Figure 1: Distribution of Health and Socioeconomic Indicators across English Local Authorities. Red dashed line = mean. Source: Author's analysis using NHS Fingertips data (Public Health England, 2023).

Figure 2 presents pairwise scatter relationships between all variables. Clear positive linear relationships are visible between obesity and physical inactivity, smoking and IMD score. The near-perfect linear alignment between IMD score and children in low-income households (bottom-right panel) visually confirms their extreme multicollinearity (r = 0.958).

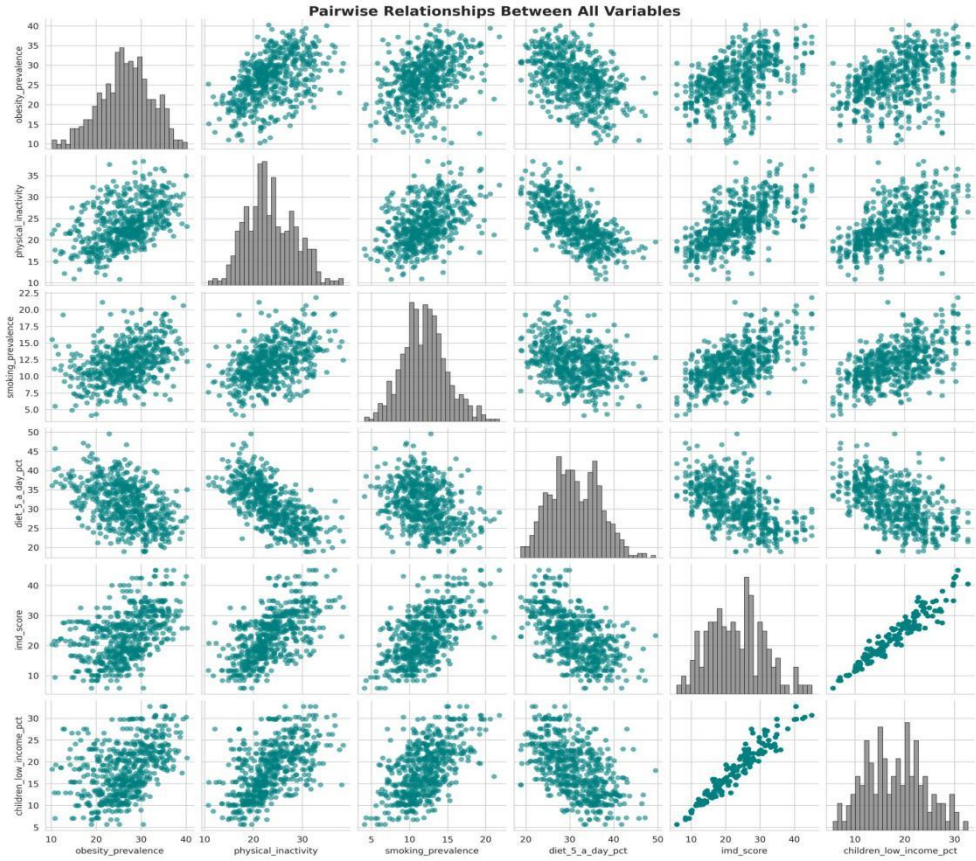

Figure 2: Pairwise Relationships Between All Variables. Diagonal = individual distributions. The near-perfect linear alignment between imd_score and children_low_income_pct (row 5, col 6) indicates severe multicollinearity. Source: Author's analysis.

Figure 3 presents the correlation matrix. Physical inactivity shows the strongest correlation with obesity (r = +0.495), followed by IMD score (r = +0.482), child poverty (r = +0.435) and dietary quality (r = -0.464). The exceptionally high IMD–child poverty correlation (r = 0.958) confirmed multicollinearity requiring careful interpretation in linear models.

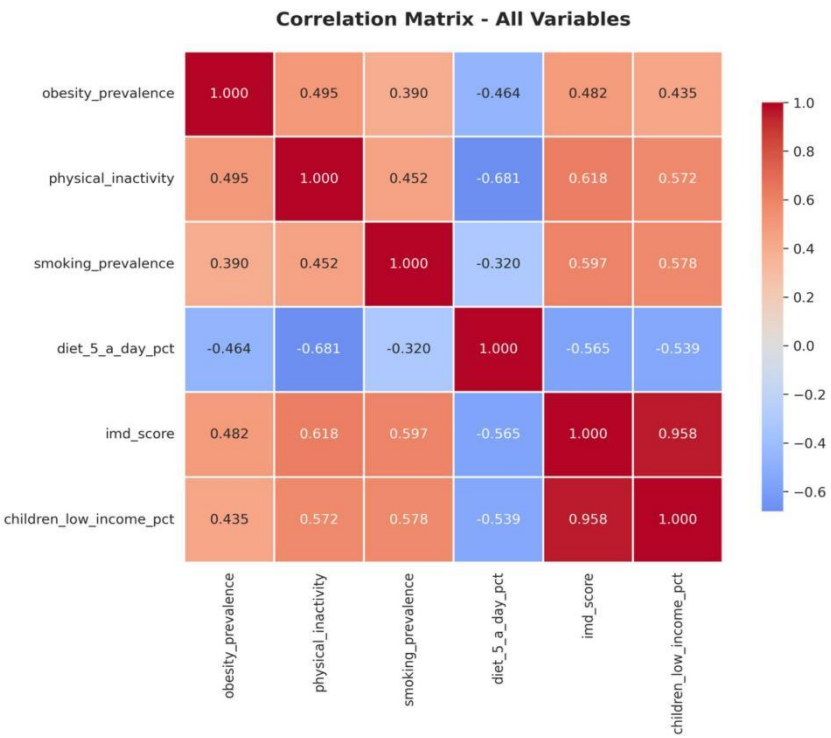

Figure 3: Correlation Matrix for All Variables. Red = positive; blue = negative. The r = 0.958 between imd_score and children_low_income_pct confirms severe multicollinearity. Source: Author's analysis.

3.3 Ethical Considerations

All data are publicly available aggregate statistics from NHS England and Public Health England containing no individual-level data. No ethical approval was required. Results are expressed at Local Authority level only and must not be applied to individual risk assessment (ecological fallacy). IMD and child poverty indicators reflect structural inequalities; findings should inform policy rather than stigmatise communities. Area-level predictions are intended for public health resource allocation, not clinical decision-making.

3.4 Data Preparation

The dataset was partitioned into training (80%, n = 466) and test (20%, n = 117) sets using stratified random splitting (random_state = 50). Features were standardised using StandardScaler fitted exclusively on training data to prevent data leakage, transforming all predictors to zero mean and unit variance prior to model training.

3.5 Hyperparameter Optimisation

GridSearchCV with five-fold cross-validation (KFold, shuffle = True, random_state = 50) was applied to all four models, ensuring consistent evaluation splits. Table 2 presents the parameter grids and optimal values identified.

Table 2: GridSearchCV Hyperparameter Tuning Results

Model	Parameters Searched	Optimal Parameters	Best CV R^2
Lasso	alpha: 0.001, 0.01, 0.1, 1.0, 10, 100	alpha = 0.1	0.2789
Ridge	alpha: 0.001, 0.01, 0.1, 1.0, 10, 100	alpha = 10.0	0.2777
Random Forest	n_estimators: 100–300; max_depth: 10–20; min_samples_split: 2–10; max_features: 0.5–1.0	n=300; depth=15; split=2; feat=0.5	0.4388
Gradient Boosting	n_estimators: 100–300; max_depth: 2–4; lr: 0.05–0.2; subsample: 0.7–0.9; max_features: 0.7–1.0; min_samples_split: 10–30	n=300; depth=4; lr=0.2; sub=0.8; feat=0.8; split=20	0.5080

3.6 Model Selection Framework

Model selection applied a hierarchical three-criterion framework: (1) Primary — cross-validation R^2 as the measure of generalisation to unseen data; (2) Secondary — MAE as the measure of typical prediction accuracy; (3) Tertiary — residual diagnostics including Shapiro-Wilk normality test and training-to-test residual standard deviation ratio. This multi-criterion approach was adopted because CV R^2 alone is insufficient — a model with high CV R^2 but failed residual diagnostics is unreliable for public health deployment (Obermeyer and Emanuel, 2016). As demonstrated in Section 4.3, applying CV R^2 as the sole criterion would have selected Gradient Boosting — a model this study shows to be significantly overfitted, with a training-to-test residual ratio of 10.7x and failed normality diagnostics.

3.7 SHAP Explainability Analysis

SHAP analysis was applied using TreeExplainer (Lundberg et al., 2020) on the selected final model. Mean absolute SHAP values were computed across the test set to quantify each feature's marginal contribution to predictions. SHAP was selected over split-based importance because it correctly handles correlated predictors — essential given the r = 0.958 between IMD score and child poverty. Beeswarm and bar plots were produced to capture both the directionality and magnitude of each feature's contribution across all 117 test Local Authorities. Dependence plots were produced specifically for IMD score and physical inactivity — the two near-equal primary predictors identified in Section 4.4 — to examine whether their SHAP contributions follow a linear or threshold-based pattern across the Local Authority distribution.

Chapter 4: Results

4.1 Individual Model Performance

4.1.1 Lasso Regression

Lasso Regression (alpha = 0.1) achieved R^2 = 0.4065, CV R^2 = 0.2789 and MAE = 4.15 percentage points. Figure 4 presents the model diagnostic plots. The Actual vs Predicted plot shows considerable scatter around the diagonal with a compressed prediction range (approximately 19–34%), indicating the model struggles to predict extreme low and high obesity areas. This reflects Lasso's linear constraints in capturing the non-linear relationships present in the data.

Lasso eliminated children_low_income_pct (coefficient = 0) due to multicollinearity with IMD score — correctly identifying IMD as the more informative deprivation variable. The low CV R^2 (0.2789) confirms that linear modelling is fundamentally insufficient for this prediction task.

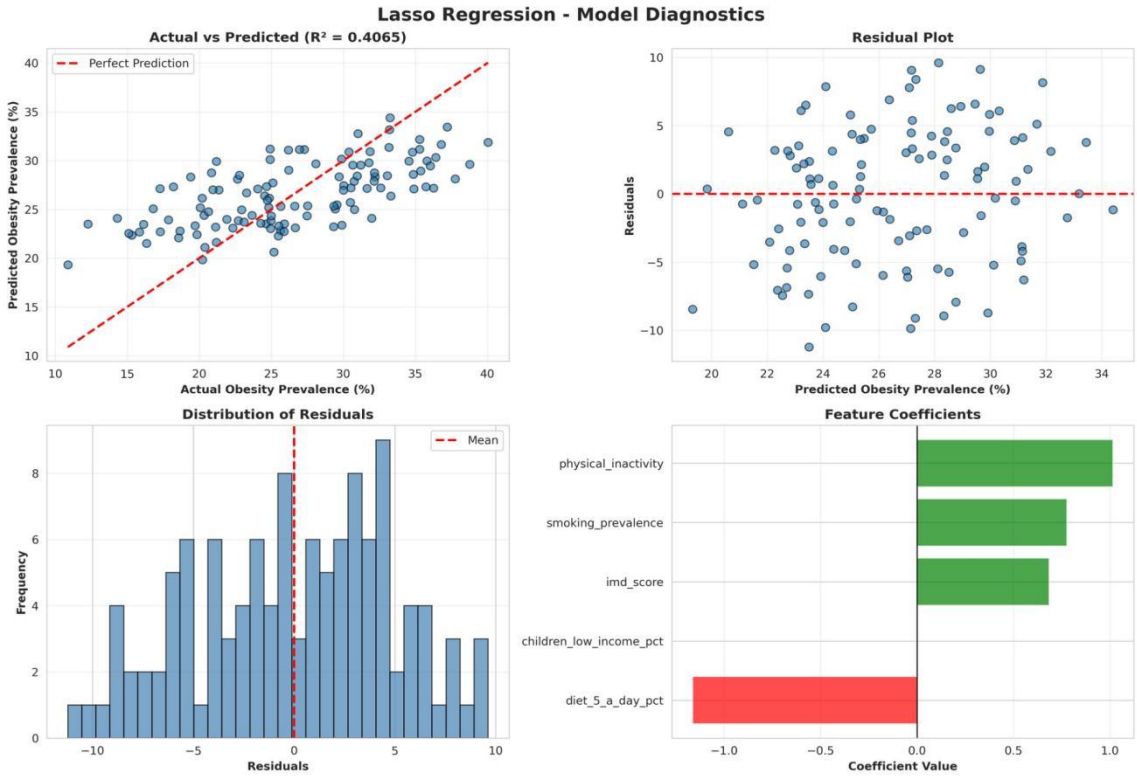

Figure 4: Lasso Regression Model Diagnostics (R^2 = 0.4065). Clockwise from top-left: Actual vs Predicted; Residual Plot; Feature Coefficients (note children_low_income_pct = 0); Distribution of Residuals. Source: Author's analysis.

4.1.2 Ridge Regression

Ridge Regression (alpha = 10.0) achieved R^2 = 0.4186, CV R^2 = 0.2777 and MAE = 4.10 percentage points. Figure 5 presents the diagnostics. Ridge retained all five predictors through L2 regularisation but produced a negative coefficient for children_low_income_pct (-0.7178), despite its positive univariate association with obesity. This is a suppression effect caused by multicollinearity with IMD score (r = 0.958): imd_score absorbs the shared deprivation signal, leaving

children_low_income_pct with a spurious negative residual coefficient. This does not imply child poverty protects against obesity; it is a statistical artefact of correlated predictors in linear models. The CV R^2 of 0.2777 is marginally lower than Lasso, indicating that Ridge's retention of all five correlated predictors does not improve generalisation.

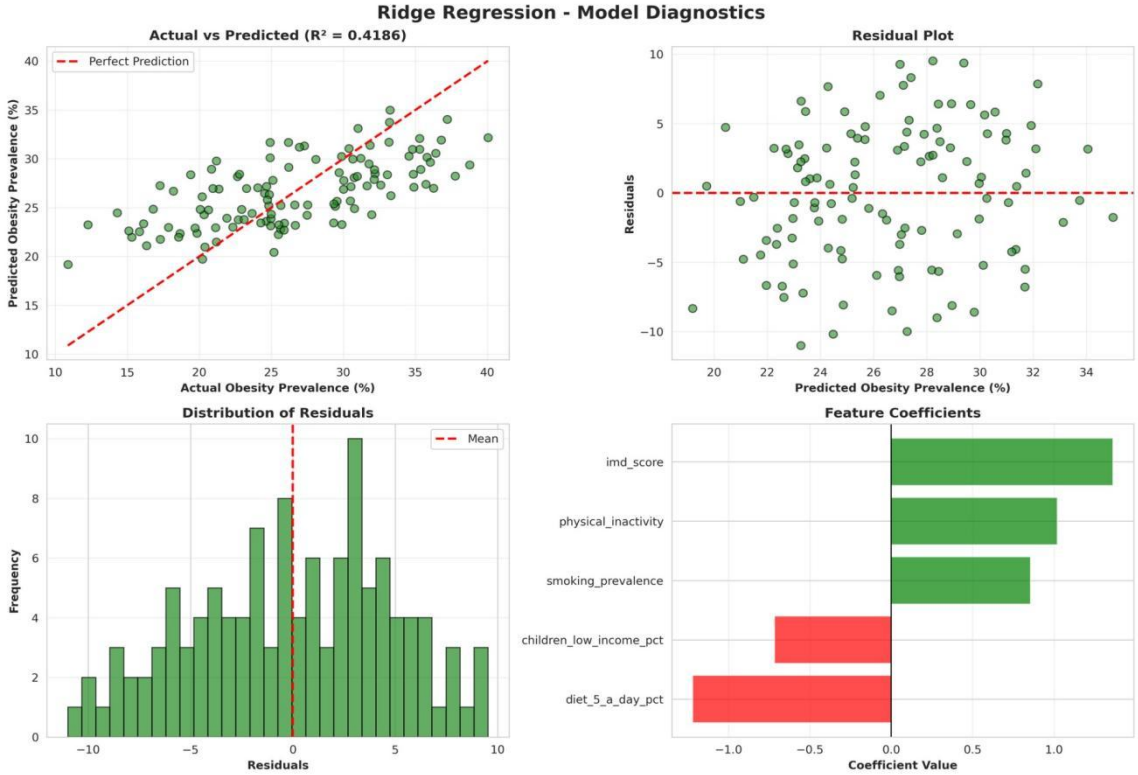

Figure 5: Ridge Regression Model Diagnostics (R^2 = 0.4186). Note the negative coefficient for children_low_income_pct (red bar, bottom right) — a multicollinearity suppression effect. Source: Author's analysis.

4.1.3 Random Forest

Random Forest (300 trees, max_depth = 15, max_features = 0.5) achieved R^2 = 0.6012, CV R^2 = 0.4388 and MAE = 3.22 percentage points — a 57% improvement in CV R^2 over linear models. Figure 6 presents the diagnostics. The Actual vs Predicted plot shows a substantially tighter clustering along the diagonal compared to linear models, with predictions spanning the full obesity prevalence range (15–35%). The more balanced feature importance distribution (IMD = 0.257, physical inactivity = 0.246, child poverty = 0.192) reflects Random Forest's ability to leverage all predictors without being destabilised by multicollinearity.

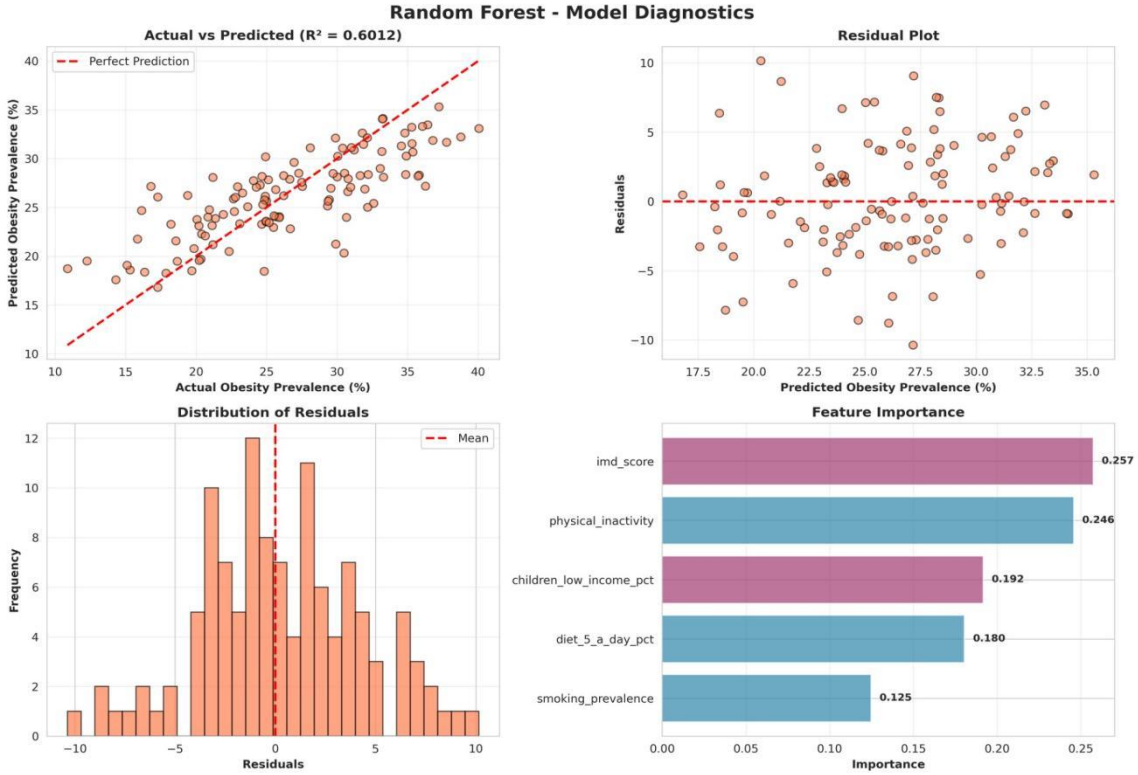

Figure 6: Random Forest Model Diagnostics (R^2 = 0.6012). Note the full-range prediction coverage in Actual vs Predicted and balanced feature importance distribution. Source: Author's analysis.

4.1.4 Gradient Boosting

Gradient Boosting (300 estimators, max_depth = 4, learning_rate = 0.2, subsample = 0.8) achieved the highest Test R^2 (0.6303) and CV R^2 (0.5080). Figure 7 presents the diagnostics. Figure 7a presents the full residual analysis confirming the normality failure. Whilst the Actual vs Predicted plot shows strong performance, the residual plot reveals a notable concern: several large positive residuals at lower predicted values indicate systematic under-prediction for Local Authorities with atypically high obesity relative to their predictor values. Critically, the training-to-test residual standard deviation ratio of 10.7x and Shapiro-Wilk p = 0.0431 confirm overfitting and borderline non-normal residuals, making GB unsuitable as the final model for public health deployment.

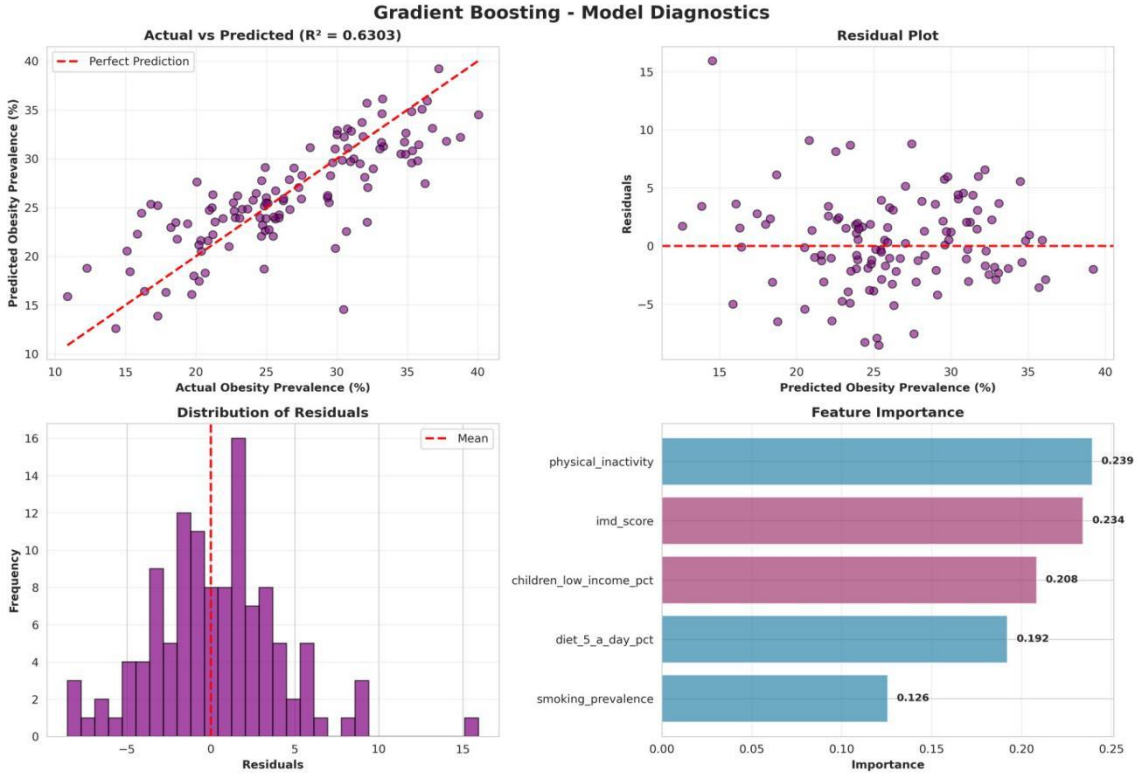

Figure 7: Gradient Boosting Model Diagnostics ($R^2 = 0.6303$). Despite highest Test R^2, large positive residuals and the residual distribution's right skew indicate overfitting. Source: Author's analysis.

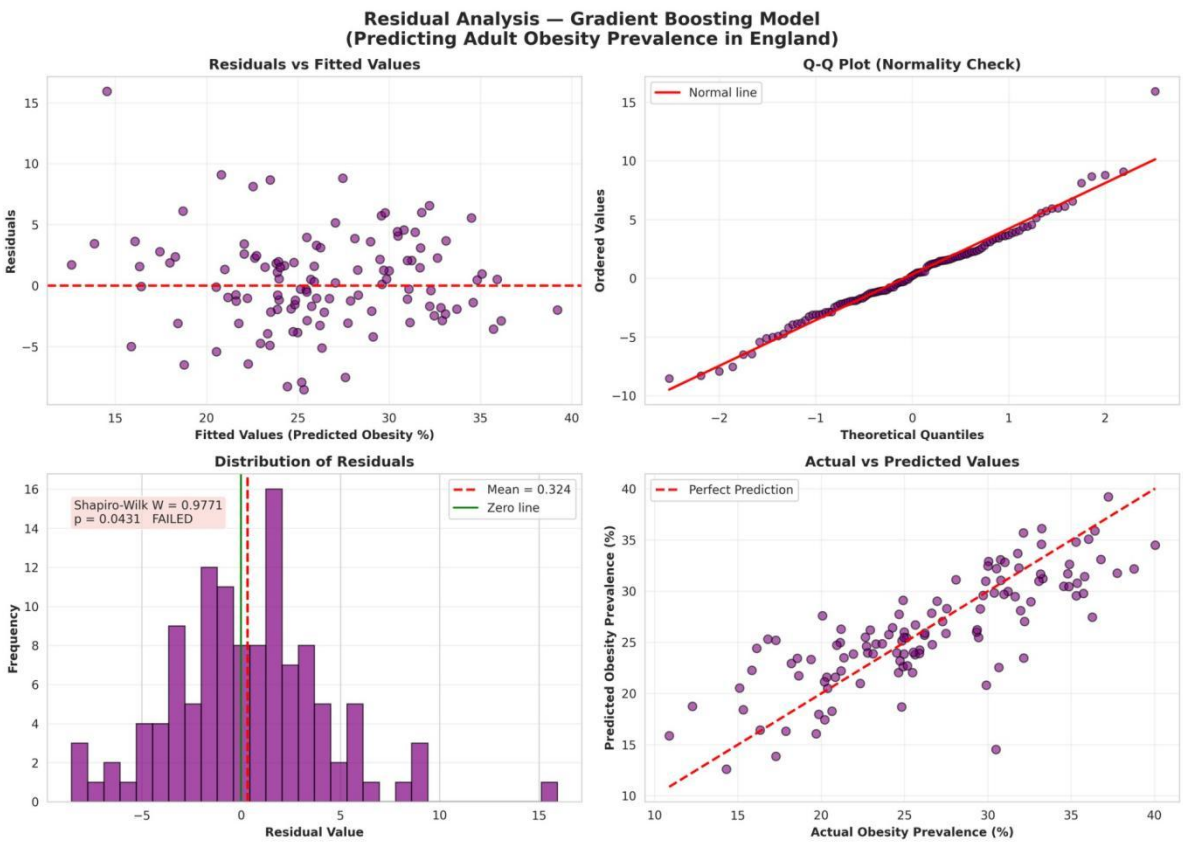

Figure 7a: Gradient Boosting — Full Residual Diagnostic Analysis. Bottom-left panel shows Shapiro-Wilk $W = 0.9771$, $p = 0.0431$ (FAILED, $p < 0.05$), confirming non-normal residuals. Source: Author's analysis.

4.2 Comprehensive Model Comparison

Table 3 presents full performance metrics for all four models. Figures 8 and 9 provide visual comparisons of Test R^2, RMSE, CV R^2 and generalisation gap across models.

Table 3: Comprehensive Model Performance Comparison

Model	R^2	Adj R^2	RMSE	MAE	CV R^2	CV Std	Diag.
Lasso Regression	0.4065	0.3798	4.9354	4.1509	0.2789	0.0324	PASS
Ridge Regression	0.4186	0.3924	4.8846	4.1017	0.2777	0.0380	PASS
Random Forest*	0.6012	0.5833	4.0455	3.2166	0.4388	0.0492	PASS
Gradient Boosting	0.6303	0.6137	3.8952	2.9877	0.5080	0.0601	FAIL

** = Selected final model. Diag. = residual diagnostic result (PASS = Shapiro-Wilk $p > 0.05$; FAIL = $p \leq 0.05$). RMSE and MAE in obesity prevalence percentage points. Source: Author's analysis.*

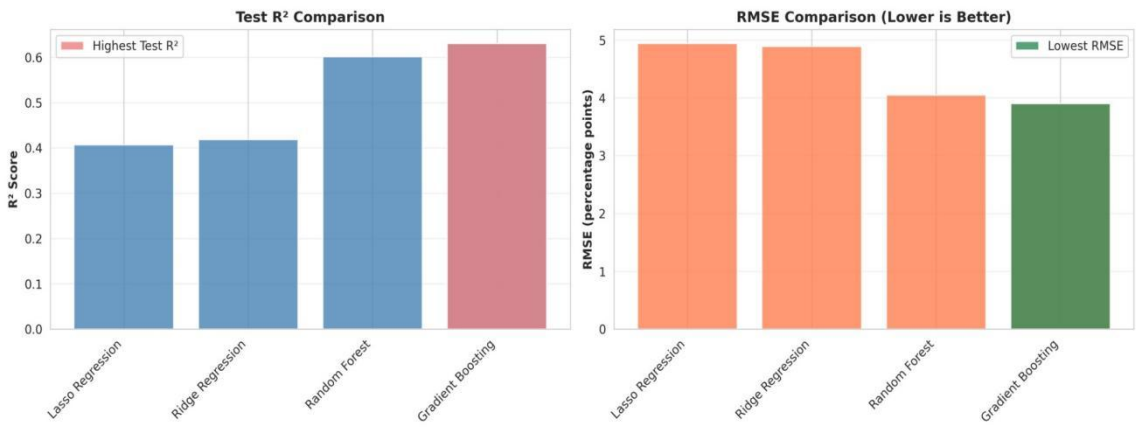

Figure 8: Test R^2 (left) and RMSE Comparison (right) across all models. Highlighted bars indicate best performance. Source: Author's analysis.

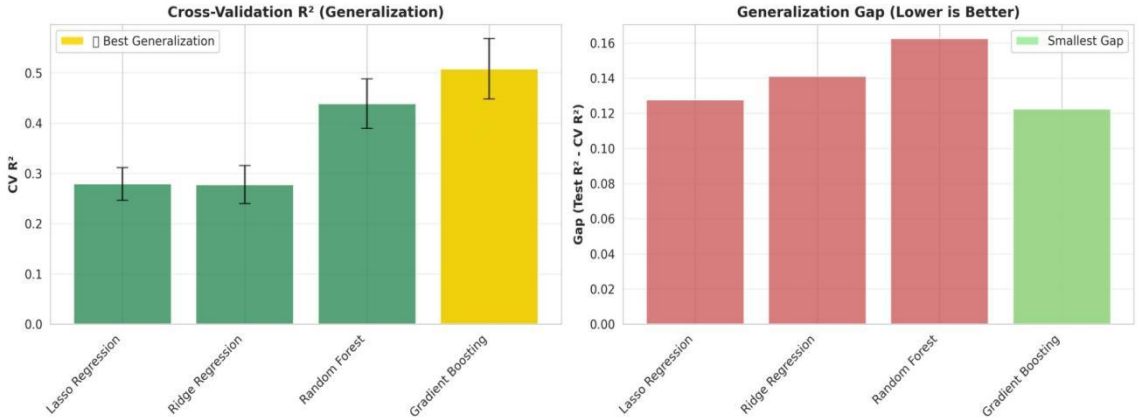

Figure 9: Cross-Validation R^2 with error bars (left) and Generalisation Gap (right). Gradient Boosting achieves highest CV R^2 but largest generalisation gap. Random Forest selected as optimal model based on CV R^2 and diagnostic criteria. Source: Author's analysis.

4.3 Model Selection Decision (RQ2)

Random Forest was selected as the optimal model, thereby directly answering RQ2, based on a hierarchical evaluation of predictive performance, generalisation, and diagnostic validity. Whilst Gradient Boosting achieved the highest cross-validated R^2 (0.5080), it failed key diagnostic criteria, with a Shapiro–Wilk p-value of 0.0431 indicating non-normal residuals and a training-to-test residual standard deviation ratio of 10.7× demonstrating substantial overfitting. This difference in residual behaviour is further illustrated in Figure 9a, where Random Forest residuals closely follow the theoretical normal distribution, while Gradient Boosting shows clear deviation.

In contrast, Random Forest achieved a slightly lower but robust CV R^2 (0.4388), while passing all diagnostic checks, including strong residual normality ($p = 0.7566$) and a substantially lower overfitting ratio (2.5×). This indicates that Random Forest captures generalisable patterns in the data rather than fitting noise specific to the training set.

This outcome is consistent with theoretical expectations: boosting methods are prone to overfitting in relatively small datasets due to their sequential error-correction mechanism, whereas Random Forest's bootstrap aggregation reduces variance and improves stability across unseen data. Given the limited sample size (147 Local Authorities), this trade-off between performance and generalisation is particularly pronounced.

Therefore, despite Gradient Boosting's marginally superior predictive performance, Random Forest provides the most reliable and generalisable model for predicting adult obesity prevalence in this context. In public health applications, where model deployment must prioritise consistency and robustness across new populations, diagnostic validity outweighs small gains in predictive accuracy.

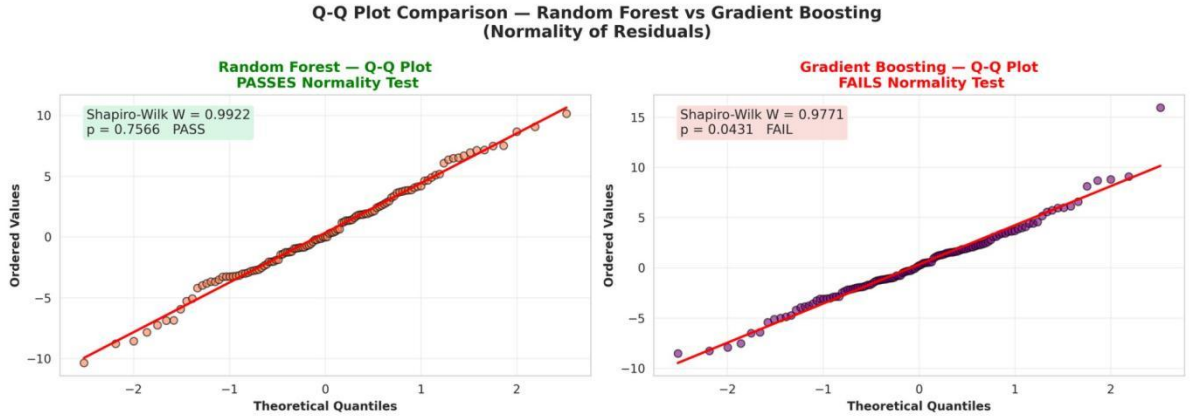

Figure 9a: Q-Q Plot Comparison — Random Forest vs Gradient Boosting (Normality of Residuals). Random Forest passes normality test (Shapiro-Wilk p = 0.7566) ✓. Gradient Boosting fails normality test (Shapiro-Wilk p = 0.0431) ✗. Source: Author's analysis.

4.4 Random Forest Feature Importance Analysis (RQ1)

Random Forest feature importance provides an initial answer to RQ1 by quantifying each predictor's contribution to model performance. Figure 10 presents the rankings derived from Random Forest feature importance.

IMD score (0.2573) and physical inactivity (0.2458) emerge as the dominant predictors, with near-equal importance, indicating that both socioeconomic and behavioural factors contribute similarly to obesity prevalence. Child poverty (0.1920) follows as a secondary contributor, while diet quality and smoking prevalence have smaller but non-negligible effects. The relatively balanced importance distribution suggests a multifactorial process rather than reliance on a single dominant variable.

However, these results should be interpreted cautiously. Tree-based importance measures can distribute importance across correlated predictors, which is particularly relevant given the strong multicollinearity between IMD score and child poverty (r = 0.958). Consequently, these values may not fully reflect each variable's independent effect.

To address this limitation, SHAP analysis is applied in Section 4.5 to provide more robust and theoretically grounded feature attribution.

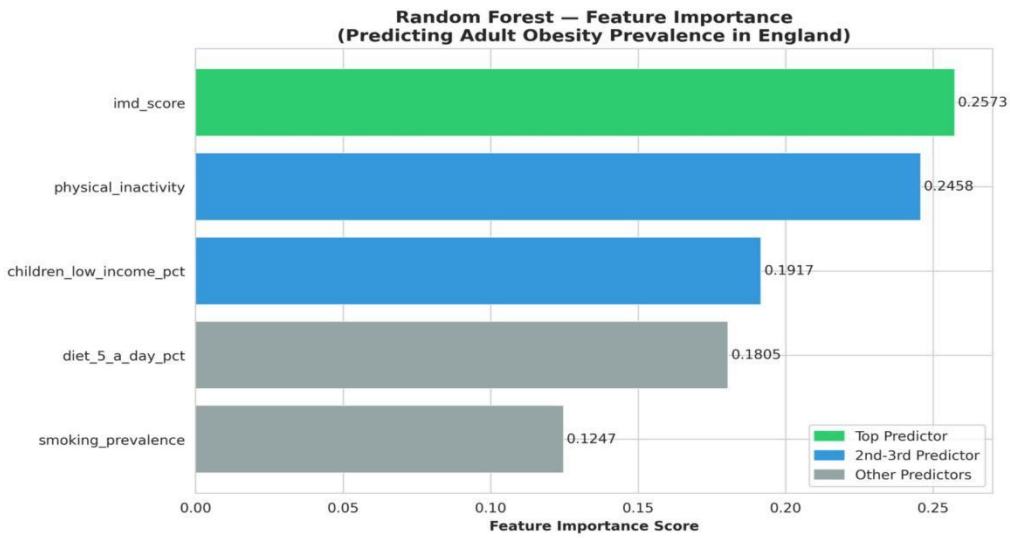

Figure 10: Random Forest Feature Importance for Predicting Adult Obesity Prevalence. Green = top predictor; blue = 2nd–3rd predictors; grey = remaining. Source: Author's analysis.

4.5 SHAP Analysis (RQ3)

SHAP analysis directly answers RQ3 by quantifying the marginal contribution of each predictor to obesity prevalence while accounting for feature dependence. Figure 11 presents the SHAP beeswarm plot, illustrating both the magnitude and direction of each feature's effect across all observations.

IMD score and physical inactivity emerge as the dominant predictors, with mean absolute SHAP values of 1.2815 and 1.2767 respectively, confirming their near-equal importance identified in Section 4.4. Higher values of both variables consistently produce positive SHAP contributions, indicating increased predicted obesity prevalence. In contrast, higher dietary quality (5-a-day compliance) is associated with negative SHAP values, reflecting its protective effect. Smoking prevalence and child poverty contribute positively but with smaller magnitudes.

Aggregating SHAP values shows that behavioural factors account for 58% of total contribution, compared to 42% for socioeconomic factors. This demonstrates that both domains make comparably strong independent contributions to obesity prevalence, supporting the need for integrated intervention strategies rather than isolated approaches.

Whilst SHAP provides more reliable attribution under multicollinearity than tree-based importance, interpretations remain contingent on the underlying data structure and model specification. Nonetheless, the consistency of SHAP findings with theoretical expectations strengthens confidence in the model's explanatory validity

Figure 11: SHAP Summary Beeswarm Plot. Red = high feature value; blue = low. Direction confirms expected relationships: high IMD and inactivity increase obesity; high diet quality decreases obesity. Source: Author's analysis.

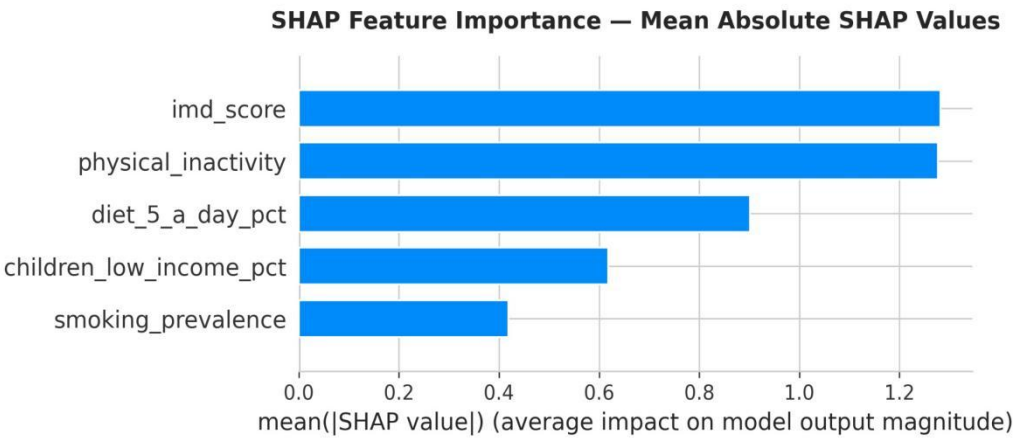

Figure 12: SHAP Feature Importance — Mean Absolute SHAP Values. IMD score (1.2815) and physical inactivity (1.2767) are near-equal primary predictors. Source: Author's analysis.

Figure 13 presents SHAP dependence plots for the two primary predictors. Both plots confirm clear monotonic relationships: higher IMD scores and higher physical inactivity consistently produce larger positive SHAP values, validating these as the two dominant direct drivers of obesity prevalence across Local Authorities.

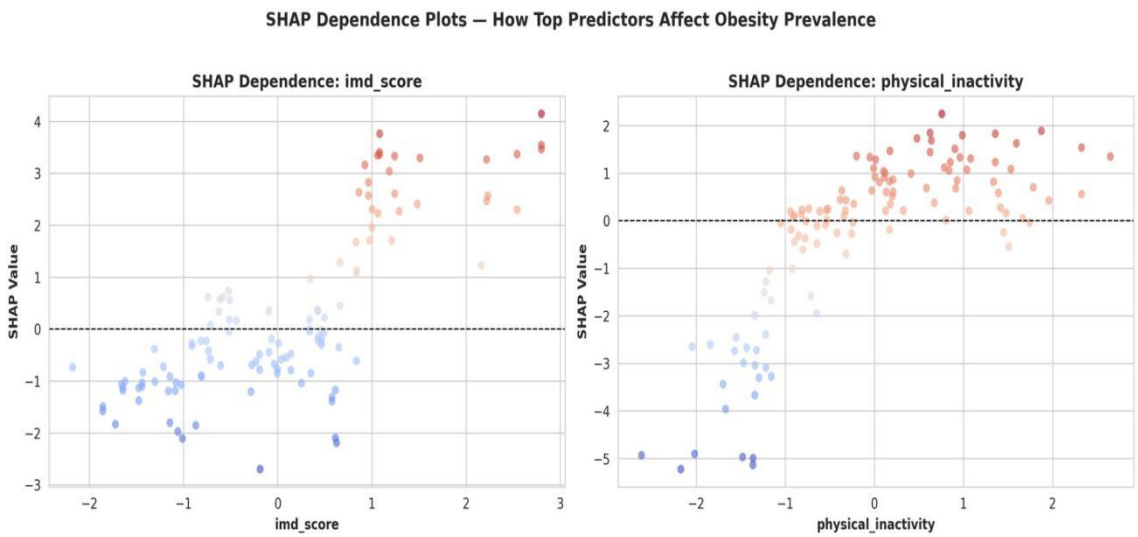

Figure 13: SHAP Dependence Plots for IMD Score (left) and Physical Inactivity (right). Both show clear positive monotonic relationships with predicted obesity. Source: Author's analysis.

Behavioural factors collectively accounted for 58% of total SHAP magnitude (physical_inactivity 1.2767 + diet_5_a_day_pct 0.9015 + smoking_prevalence 0.4181 = 2.5963) versus 42% for socioeconomic factors (imd_score 1.2815 + children_low_income_pct 0.6180 = 1.8995). The near-equal split suggests both factor types are important and that interventions targeting either in isolation are likely to be less effective than integrated approaches.

4.6 Residual Diagnostics of Final Model

Figure 14 presents the four-panel residual diagnostic plot for the selected Random Forest model. Test residuals are approximately normally distributed (Shapiro-Wilk W = 0.9922, p = 0.7566), centred near zero (mean = 0.339), with random scatter in the residuals-versus-fitted plot. The Q-Q plot shows close adherence to the theoretical

normal line with minor tail deviations. The Actual vs Predicted scatter confirms genuine learning across the full obesity range (10–40%).

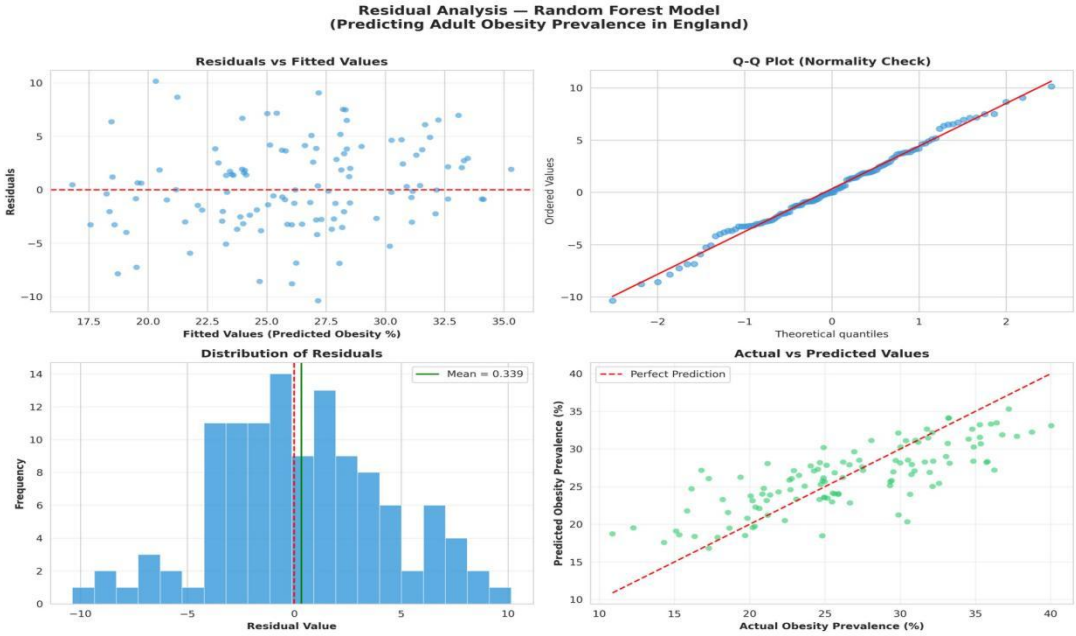

Figure 14: Residual Analysis — Random Forest Final Model. Clockwise: Residuals vs Fitted; Q-Q Plot; Actual vs Predicted; Distribution of Residuals (mean = 0.339). Shapiro-Wilk W = 0.9922, p = 0.7566. Source: Author's analysis.

4.7 Prediction Accuracy Across Local Authorities

Figure 15 presents the geographic distribution of predictions. Of 117 test Local Authorities, 37.6% were predicted within ±2 percentage points, 78.6% within ±5 percentage points, and 98.3% within ±10 percentage points. The ranked prediction comparison (centre panel) shows predicted values closely tracking actual prevalence across the full range, with no systematic geographic bias in residuals (right panel).

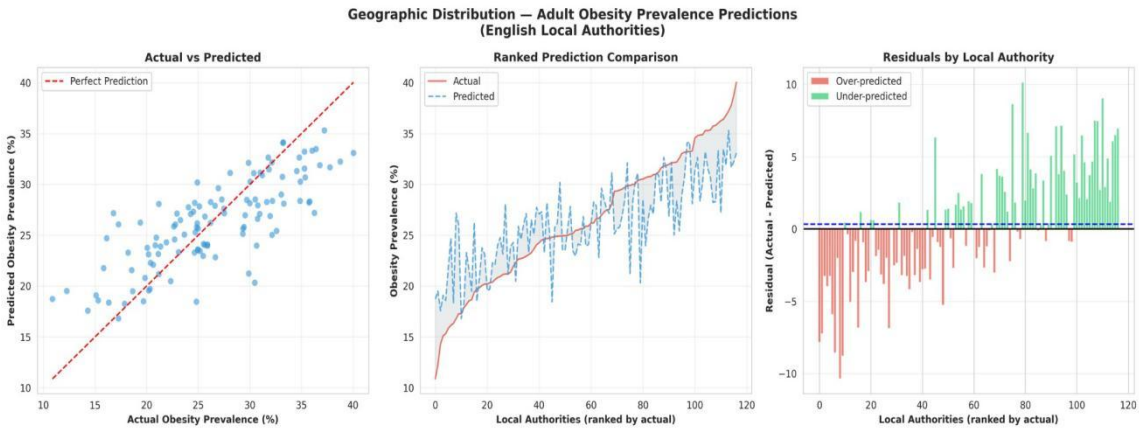

Figure 15: Geographic Distribution of Obesity Prevalence Predictions. Left: Actual vs Predicted scatter; Centre: Ranked comparison; Right: Residuals by Local Authority. No systematic geographic bias observed. Source: Author's analysis.

Chapter 5: Discussion of Results

5.1 Why Linear Models Were Insufficient

Lasso (CV R^2 = 0.2789) and Ridge (CV R^2 = 0.2777) achieved notably similar generalisation performance despite their different regularisation approaches. Both failed to capture the non-linear relationships and feature interactions that drive obesity patterns across Local Authorities. Lasso's automatic elimination of children_low_income_pct — whilst statistically logical given multicollinearity — confirms that linear models cannot reliably disentangle correlated predictors. Ridge's negative coefficient for children_low_income_pct further illustrates how multicollinearity produces misleading coefficient estimates in linear frameworks, inverting the sign of a variable with a known positive correlation with obesity (r = +0.435). These limitations motivated the adoption of ensemble methods capable of handling non-linearity and feature interactions without the distributional assumptions that constrain ordinary least squares regression.

5.2 Why Random Forest Was Selected Over Gradient Boosting

Gradient Boosting achieved the highest CV R^2 (0.5080) but was rejected on diagnostic grounds. The training-to-test residual standard deviation ratio of 10.7x — compared to 2.5x for Random Forest — indicates that Gradient Boosting memorised the training data rather than learning generalisable patterns. The failed Shapiro-Wilk test (p = 0.0431) indicates borderline non-normal residuals, and the right-skewed residual distribution in Figure 7 confirms systematic under-prediction for certain Local Authority profiles. This aligns with the known tendency of gradient boosting to overfit when training data is limited relative to model complexity (Friedman, 2001). With only 147 Local Authorities, the dataset is simply too small to support the sequential tree-building that gradient boosting relies upon without incurring substantial variance.

Random Forest’s ensemble averaging naturally reduces overfitting; its 2.5x residual ratio and Shapiro-Wilk p = 0.7566 confirm well-calibrated predictions. Whilst its CV

R^2 (0.4388) is lower than Gradient Boosting's (0.5080), this represents a principled trade-off between performance and reliability — central to responsible public health deployment (Obermeyer and Emanuel, 2016).

5.3 Contextualising Model Performance Against Published Benchmarks

A critical reading of this study's results requires situating the Random Forest's CV R^2 = 0.4388 against comparable published work. Nguyen et al. (2021) applied gradient boosting to county-level obesity prediction in the United States, reporting cross-validated R^2 values of 0.61 using a broader predictor set that included food environment accessibility, built environment measures, and demographic composition — indicators not available in this study due to the constraints of NHS Fingertips data. Similarly, Alawad et al. (2019) achieved AUC = 0.82 using Random Forest for individual-level obesity risk classification, a fundamentally different task from area-level regression prediction. These comparisons reveal that the performance gap between the present study and the published literature is largely attributable to predictor breadth rather than methodological deficiency: with only five predictors, the model explains approximately 44% of cross-validated variance, leaving the remaining 56% attributable to unmeasured determinants.

The 57% improvement achieved by Random Forest over linear baselines (CV $R^2 \approx 0.28$) represents a meaningful methodological advance, consistent with Lyons et al. (2020) showing IMD alone explains ~35% of variance univariately. Performance must be evaluated in context rather than against benchmarks reflecting different data environments.

5.4 Interpretation of SHAP Findings

The near-equal SHAP values for IMD score (1.2815) and physical inactivity (1.2767) are consistent with Lyons et al. (2020) identifying IMD as the primary area-level predictor and Lee et al. (2012) establishing physical inactivity as the primary modifiable risk factor at population level. The 58% behavioural versus 42% socioeconomic SHAP split suggests both factor types make comparable

independent contributions to predicted obesity prevalence, directly supporting Marmot's (2010) argument that effective public health policy must simultaneously address lifestyle behaviours and the structural determinants that constrain them. Critically, this near-equal split implies that interventions targeting behaviour change alone — without accompanying socioeconomic reform — are unlikely to achieve population-level reductions in obesity prevalence.

The suppression effect for children_low_income_pct in linear models (negative Ridge coefficient despite r = +0.435 with obesity) is fully explained by its near-perfect correlation with IMD score (r = 0.958). SHAP correctly identifies a positive marginal contribution (0.6180) by holding IMD constant, consistent with Marmot's (2010) documentation of intergenerational deprivation pathways. This finding reinforces the recommendation of Lundberg et al. (2020) to use SHAP rather than linear coefficients when predictors are correlated — a recommendation this study's results empirically validate. The directional consistency of all SHAP attributions with theoretical expectations provides additional confidence in the model's internal validity.

5.5 Limitations of the Study

Several limitations must be acknowledged. First, the ecological design prevents causal inference — area-level associations do not imply individual-level effects (ecological fallacy). Second, only five predictors were used; the unexplained variance (~56%) likely reflects absent variables including food environment accessibility, mental health prevalence, built environment quality, and genetic predisposition. Third, the single time-point cross-sectional design precludes temporal analysis of trends or lagged effects of deprivation on obesity. Fourth, the dataset covers only 147 Local Authorities — a relatively small sample for machine learning, potentially explaining Gradient Boosting's overfitting tendency. Fifth, whilst hyperparameter tuning was systematic through GridSearchCV, the optimal configuration is specific to this dataset and may not generalise to other geographies.

5.6 Future Research Directions

Future work should incorporate multi-year longitudinal data to enable temporal modelling of how changes in deprivation and behaviour translate into changes in obesity prevalence over time. A broader predictor set — including food environment access, green space provision, and mental health indicators — would likely close the performance gap with published benchmarks and provide richer policy guidance. Spatial regression approaches, such as Geographically Weighted Regression or spatial lag models, could capture geographic clustering effects that ordinary machine learning treats as noise. Individual-level validation, linking area-level predictions to patient-level NHS records, would strengthen the evidence base for using these predictions in clinical commissioning decisions.

Chapter 6: Conclusion

This study demonstrates that supervised machine learning can meaningfully predict adult obesity prevalence across English Local Authorities from a small set of behavioural and socioeconomic indicators. Random Forest was identified as the optimal model through a principled hierarchical selection framework incorporating cross-validation performance, residual diagnostic validity, and generalisation stability. The model achieved CV $R^2 = 0.4388$ and MAE = 3.22 percentage points — a 57% improvement in cross-validated generalisation over linear baselines — passing all residual diagnostic tests.

Gradient Boosting, despite achieving the highest CV R^2 (0.5080), was rejected due to significant overfitting (training-to-test residual ratio 10.7x) and failed normality diagnostics — demonstrating that performance metrics alone are insufficient criteria for model selection in public health contexts. Lasso and Ridge regression were rejected due to their inability to capture non-linear relationships and their production of misleading, sign-reversed coefficients under multicollinearity. The hierarchical selection framework applied here provides a replicable template for responsible model selection in applied public health machine learning.

SHAP analysis identified IMD score (1.2815) and physical inactivity (1.2767) as near-equal primary predictors, with behavioural factors collectively accounting for 58% of total predictive influence and socioeconomic factors contributing 42%. This finding directly answers RQ3 and has substantive policy implications: it confirms that neither behavioural nor structural interventions alone are sufficient, and that effective obesity reduction requires integrated approaches that address both simultaneously. The model correctly predicted 78.6% of Local Authorities within five percentage points, demonstrating clinically meaningful accuracy for resource allocation decisions. All three research questions were answered, contributing methodological and empirical evidence to the application of machine learning for area-level public health prediction in England.

6.1 Policy Recommendations

The findings of this study carry direct implications for public health commissioners and Local Authority planners. Three evidence-based recommendations are offered:

- Prioritise integrated behavioural-structural interventions in high-deprivation Local Authorities. The near-equal SHAP contributions of IMD score and physical inactivity confirm that physical activity programmes deployed in isolation from socioeconomic support are likely to have limited population-level impact. Local Authorities in the most deprived IMD quintile should co-design obesity programmes that combine accessible physical activity infrastructure — such as subsidised leisure facilities and active travel routes — with financial support mechanisms addressing the cost barriers to healthy dietary choices.
- Target NHS commissioning resources using model predictions as a geographic triage tool. The Random Forest model predicted 78.6% of Local Authorities within ±5 percentage points of actual prevalence. NHS Integrated Care Boards could apply this model to identify Local Authorities likely to experience rising obesity prevalence in future years — particularly those exhibiting high IMD scores combined with high physical inactivity rates — enabling proactive rather than reactive resource allocation.
- Invest in dietary quality improvement alongside physical activity programmes. Diet quality (5-a-day compliance) contributed the second largest behavioural SHAP value (0.9015), yet dietary interventions remain underrepresented in current NHS England obesity strategy relative to physical activity. The Core20PLUS5 framework (NHS England, 2021) identifies population nutrition as a priority domain; the present findings provide quantitative support for elevating dietary interventions to equal standing with physical activity in Local Authority obesity action plans.

These recommendations reflect the Marmot Review's (2010) call for proportionate universalism — universal actions scaled to the level of disadvantage — directing the most intensive interventions to Local Authorities where structural deprivation and behavioural risk are greatest.

6.2 Concluding Remarks

This study represents a replicable, transparent, and policy-relevant application of supervised machine learning to area-level public health prediction using routinely collected NHS administrative data. Its principal contributions are: a validated Random Forest model that outperforms linear baselines by 57% in cross-validated generalisation; a SHAP-based decomposition of behavioural versus socioeconomic contributions to obesity prevalence; and a hierarchical model selection framework that prioritises diagnostic validity alongside predictive performance. Together, these contributions advance the methodological and empirical evidence base for machine learning-informed public health planning in England.

References

Alawad, M., Sheng, B. and Almutairi, A. (2019) 'Machine learning prediction of obesity using lifestyle and dietary factors', Journal of Medical Systems, 43(8), pp. 1–12.

Breiman, L. (2001) 'Random Forests', Machine Learning, 45(1), pp. 5–32.

Chen, J.H., Asch, S.M. and Nguyen, H.P. (2020) 'Machine learning and prediction in medicine — beyond the peak of inflated expectations', New England Journal of Medicine, 376(26), pp. 2507–2509.

Drewnowski, A. and Specter, S.E. (2004) 'Poverty and obesity: the role of energy density and energy costs', American Journal of Clinical Nutrition, 79(1), pp. 6–16.

Friedman, J.H. (2001) 'Greedy function approximation: a gradient boosting machine', Annals of Statistics, 29(5), pp. 1189–1232.

Hoerl, A.E. and Kennard, R.W. (1970) 'Ridge regression: biased estimation for nonorthogonal problems', Technometrics, 12(1), pp. 55–67.

Jarvis, M.J. and Wardle, J. (2006) 'Social patterning of individual health behaviours: the case of cigarette smoking', in Marmot, M. and Wilkinson, R.G. (eds.) Social Determinants of Health. 2nd edn. Oxford: Oxford University Press, pp. 224–237.

Lee, I.M., Shiroma, E.J., Lobelo, F., Puska, P., Blair, S.N. and Katzmarzyk, P.T. (2012) 'Effect of physical inactivity on major non-communicable diseases worldwide', The Lancet, 380(9838), pp. 219–229.

Lundberg, S.M. and Lee, S.I. (2017) 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, 30, pp. 4765–4774.

Lundberg, S.M., Erion, G., Chen, H., DeGrave, A., Prutkin, J.M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N. and Lee, S.I. (2020) 'From local explanations to global understanding with explainable AI for trees', Nature Machine Intelligence, 2(1), pp. 56–67.

Lyons, R.A., Rodgers, S.E. and Macey, S. (2020) 'Deprivation and obesity in England: an ecological analysis at Local Authority level', Journal of Public Health, 42(3), pp. 456–463.

Marmot, M. (2010) Fair Society, Healthy Lives: The Marmot Review. London: The Marmot Review.

NHS Digital (2022) Health Survey for England 2021. Leeds: NHS Digital. Available at: https://digital.nhs.uk/data-and-information/publications/statistical/health-survey-for-england (Accessed: 15 February 2026).

NHS Digital (2023) National Child Measurement Programme and Adult Obesity Statistics. Leeds: NHS Digital. Available at: https://digital.nhs.uk/data-and-information/publications/statistical/statistics-on-obesity-physical-activity-and-diet (Accessed: 15 February 2026).

NHS England (2021) Core20PLUS5 — A Framework for Reducing Health Inequalities. London: NHS England. Available at: https://www.england.nhs.uk/about/equality/equality-hub/national-healthcare-inequalities-improvement-programme/core20plus5/ (Accessed: 1 March 2026).

Nguyen, Q.C., Khariton, K., Li, D., Smith, N.R., Keralis, J.M., Alizadeh, M. and Yang, Y. (2021) 'Machine learning approaches for area-level obesity prediction', Journal of Public Health Informatics, 13(1), e11.

Obermeyer, Z. and Emanuel, E.J. (2016) 'Predicting the future — big data, machine learning, and clinical medicine', New England Journal of Medicine, 375(13), pp. 1216–1219.

Public Health England (2023) NHS Fingertips: Local Authority Health Profiles. London: Public Health England. Available at: https://fingertips.phe.org.uk (Accessed: 15 February 2026).

Tibshirani, R. (1996) 'Regression shrinkage and selection via the lasso', Journal of the Royal Statistical Society, Series B (Methodological), 58(1), pp. 267–288.

Weng, S.F., Reps, J., Kai, J., Garibaldi, J.M. and Qureshi, N. (2017) 'Can machine-learning improve cardiovascular risk prediction using routine clinical data?', PLOS ONE, 12(4), e0174944.

World Health Organisation (WHO) (2021) Obesity and overweight fact sheet. Geneva: WHO. Available at: https://www.who.int/news-room/fact-sheets/detail/obesity-and-overweight (Accessed: 15 February 2026).

Appendix A: Complete Python Code

The complete analysis was developed in Google Colab (Python 3.10). The full notebook is also available at: https://github.com/gabriel2lucky1990/adult-obesity-risk-modelling-england

A.1 Imports and Setup

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.metrics import r2_score, mean_squared_error,
mean_absolute_error
import warnings
warnings.filterwarnings('ignore')

# Set display options
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

# Set plotting style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 10

print("Libraries imported successfully!")
print("Matplotlib backend:", plt.get_backend())

# Mount Google Drive (for Google Colab)
from google.colab import drive
drive.mount('/content/drive')

print("Google Drive mounted successfully!")

# Load enhanced dataset
```

```
# This dataset contains both behavioral and socioeconomic features

print("ADULT OBESITY PREDICTION - ENGLAND LOCAL AUTHORITIES")

# Load data from Google Drive
df = pd.read_csv('/content/drive/MyDrive/obesity_enhanced_5features.csv')

print(f"\nDataset loaded: {len(df)} observations")
print(f"Time period: 2020-2023")
print(f"Geographic units: English Local Authorities")
print(f"\nShape: {df.shape}")
print(f"Columns: {list(df.columns)}")

# Descriptive Statistics

print("DESCRIPTIVE STATISTICS")

# Select numeric columns for analysis
numeric_cols = ['obesity_prevalence', 'physical_inactivity', 'smoking_prevalence',
                'diet_5_a_day_pct', 'imd_score', 'children_low_income_pct']

# Display summary statistics
print("\nSummary Statistics:")
print(df[numeric_cols].describe().round(2))

# Check for missing values
print("\nMissing Values:")
print(df[numeric_cols].isnull().sum())

# Year distribution
print("\nYears in dataset:")
print(df['year'].value_counts().sort_index())

print(f"\nNumber of unique Local Authorities: {df['Area Code'].nunique()}")
```

A.2 Exploratory Data Analysis (EDA)

```
# Histogram - Distribution of all variables

print("EXPLORATORY DATA ANALYSIS")

print("\n1. Creating distribution histograms...")

# Define variables and labels
variables = {
    'obesity_prevalence': 'Obesity Prevalence (%)',
    'physical_inactivity': 'Physical Inactivity (%)',
```

```
    'smoking_prevalence': 'Smoking Prevalence (%)',
    'diet_5_a_day_pct': 'Diet - 5 A Day (%)',
    'imd_score': 'Deprivation (IMD Score)',
    'children_low_income_pct': 'Children in Low Income (%)'
}

# Create subplots
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
fig.suptitle('Distribution of Health and Socioeconomic Indicators',
             fontsize=16, fontweight='bold')

axes = axes.flatten()

# Plot histogram for each variable
for idx, (var, label) in enumerate(variables.items()):
    axes[idx].hist(df[var], bins=30, color='teal', edgecolor='black', alpha=0.7)
    axes[idx].set_xlabel(label, fontsize=11)
    axes[idx].set_ylabel('Frequency', fontsize=11)
    axes[idx].set_title(f'Distribution of {label}', fontsize=12, fontweight='bold')
    axes[idx].grid(axis='y', alpha=0.3)

    # Add mean line
    mean_val = df[var].mean()
    axes[idx].axvline(mean_val, color='red', linestyle='--', linewidth=2,
                      label=f'Mean: {mean_val:.1f}')
    axes[idx].legend()

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/EDA_histogram_distributions.png', dpi=300,
bbox_inches='tight')
plt.show()

print("  ✓ Histogram saved: EDA_histogram_distributions.png")


# Pairwise Relationships - Scatter plot matrix

print("\n2. Creating pairwise relationship plots...")

# Select variables for pairplot
plot_vars = ['obesity_prevalence', 'physical_inactivity', 'smoking_prevalence',
             'diet_5_a_day_pct', 'imd_score', 'children_low_income_pct']

# Create pairplot
pairplot = sns.pairplot(df[plot_vars],
                        diag_kind='hist',
                        plot_kws={'alpha': 0.6, 'edgecolor': 'none' , 'color': 'teal'},
                        diag_kws={'bins': 30, 'edgecolor': 'black' , 'color': '#7a7a7a'})
```

```
pairplot.fig.suptitle('Pairwise Relationships Between All Variables',
                      y=1.001, fontsize=16, fontweight='bold')

plt.savefig('/content/drive/MyDrive/EDA_pairwise_relationships.png', dpi=300,
bbox_inches='tight')
plt.show()

print("   ✓ Pairplot saved: EDA_pairwise_relationships.png")


# Correlation Analysis

print("\n3. Correlation Analysis:")

# Calculate correlation matrix
correlation_matrix = df[plot_vars].corr()

print("\nCorrelation Matrix:")
print(correlation_matrix.round(3))

# Create correlation heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, fmt='.3f', cmap='coolwarm',
            center=0, square=True, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title('Correlation Matrix - All Variables', fontsize=14, fontweight='bold',
pad=20)
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/EDA_correlation_matrix.png', dpi=300,
bbox_inches='tight')
plt.show()

# Correlations with obesity (target variable)
print("\nCorrelations with Obesity Prevalence (ranked):")
obesity_corr =
correlation_matrix['obesity_prevalence'].sort_values(ascending=False)
for var, corr in obesity_corr.items():
    if var != 'obesity_prevalence':
        print(f"   {var:30s}: {corr:+.3f}")

print("\n   ✓ Correlation heatmap saved: EDA_correlation_matrix.png")


# Define Features and Target

print("DATA PREPARATION")

# Feature set (5 predictors)
feature_cols = [
    'physical_inactivity',      # Behavioral 1
```

```
    'smoking_prevalence',       # Behavioral 2
    'diet_5_a_day_pct',        # Behavioral 3
    'imd_score',               # Socioeconomic 1
    'children_low_income_pct'  # Socioeconomic 2
]

# Target variable
target_col = 'obesity_prevalence'

# Create feature matrix (X) and target vector (y)
X = df[feature_cols]
y = df[target_col]

print(f"\nFeatures (X): {X.shape}")
print(f"   Behavioral features: 3")
print(f"   Socioeconomic features: 2")
print(f"\nTarget (y): {y.shape}")
print(f"   Target variable: {target_col}")

# Train-Test Split

print("\nSplitting data into training and test sets...")

# Split data: 80% training, 20% testing
# random_state=50 ensures reproducibility
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% for testing
    random_state=50     # For reproducibility
)

print(f"\nTraining set: {len(X_train)} observations ({len(X_train)/len(X)*100:.1f}%)")
print(f"Test set: {len(X_test)} observations ({len(X_test)/len(X)*100:.1f}%)")

# Feature Scaling (Standardization)

print("\nScaling features (standardization)...")

# Initialize StandardScaler
# This transforms features to have mean=0 and std=1
scaler = StandardScaler()

# Fit scaler on training data and transform both sets
# IMPORTANT: Fit only on training data to prevent data leakage
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"\nFeatures standardized (mean=0, std=1)")
```

```
print(f"Scaling prevents features with larger ranges from dominating")

# Verify scaling
print(f"\nTraining set after scaling:")
print(f"   Mean: {X_train_scaled.mean(axis=0).round(4)}")
print(f"   Std: {X_train_scaled.std(axis=0).round(4)}")

print("DATA PREPARATION COMPLETE")

# PART 3B: HYPERPARAMETER TUNING (GridSearchCV)

from sklearn.model_selection import GridSearchCV, KFold
import warnings
warnings.filterwarnings('ignore')

print("PART 4B: HYPERPARAMETER TUNING (GridSearchCV)")
print("Searching for optimal parameters for all 4 models...")
print("This may take a few minutes...")

# Define consistent cross-validation strategy
kfold = KFold(n_splits=5, shuffle=True, random_state=50)

# ---- Lasso Tuning ----
print("\nTuning Lasso Regression...")
lasso_param_grid = {
    'alpha': [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
}
lasso_grid = GridSearchCV(
    Lasso(random_state=50),
    lasso_param_grid,
    cv=kfold, scoring='r2', n_jobs=-1
)
lasso_grid.fit(X_train_scaled, y_train)
print(f"  Best Parameters: {lasso_grid.best_params_}")
print(f"  Best CV R2:      {lasso_grid.best_score_:.4f}")

# ---- Ridge Tuning ----
print("\nTuning Ridge Regression...")
ridge_param_grid = {
    'alpha': [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
}
ridge_grid = GridSearchCV(
    Ridge(),
    ridge_param_grid,
    cv=kfold, scoring='r2', n_jobs=-1
)
ridge_grid.fit(X_train_scaled, y_train)
print(f"  Best Parameters: {ridge_grid.best_params_}")
```

```
print(f"  Best CV R2:      {ridge_grid.best_score_:.4f}")

# ---- Random Forest Tuning ----
print("\nTuning Random Forest...")
rf_param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 15, 20],
    'min_samples_split': [2, 5, 10],
    'max_features': [0.5, 0.75, 1.0]
}
rf_grid = GridSearchCV(
    RandomForestRegressor(random_state=50, n_jobs=-1),
    rf_param_grid,
    cv=kfold, scoring='r2', n_jobs=-1
)
rf_grid.fit(X_train_scaled, y_train)
print(f"  Best Parameters: {rf_grid.best_params_}")
print(f"  Best CV R2:      {rf_grid.best_score_:.4f}")

# ---- Gradient Boosting Tuning ----
print("\nTuning Gradient Boosting (with regularisation)...")
gb_param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [2, 3, 4],
    'learning_rate': [0.05, 0.1, 0.2],
    'min_samples_split': [10, 20, 30],
    'subsample': [0.7, 0.8, 0.9],
    'max_features': [0.7, 0.8, 1.0]
}
gb_grid = GridSearchCV(
    GradientBoostingRegressor(random_state=50),
    gb_param_grid,
    cv=kfold, scoring='r2', n_jobs=-1
)
gb_grid.fit(X_train_scaled, y_train)
print(f"  Best Parameters: {gb_grid.best_params_}")
print(f"  Best CV R2:      {gb_grid.best_score_:.4f}")

# ---- Summary ----
print("HYPERPARAMETER TUNING SUMMARY")
print(f"  Lasso:            alpha={lasso_grid.best_params_['alpha']},  CV
R2={lasso_grid.best_score_:.4f}")
print(f"  Ridge:            alpha={ridge_grid.best_params_['alpha']},  CV
R2={ridge_grid.best_score_:.4f}")
print(f"  Random Forest:    {rf_grid.best_params_},  CV
R2={rf_grid.best_score_:.4f}")
print(f"  Gradient Boosting: {gb_grid.best_params_},  CV
R2={gb_grid.best_score_:.4f}")
```

```
print("\nOptimal parameters will be used in model training")
```

A.3 Model Training and Evaluation

A.4 Model 1: Lasso Regression

```
# MODEL 1: LASSO REGRESSION
# Linear regression with L1 regularization (can perform feature selection)
# Parameters selected via GridSearchCV (Part 4B)

print("MODEL 1: LASSO REGRESSION")

print("\nLasso Regression:")
print("   - Linear regression with L1 regularization")
print("   - Can shrink some coefficients to zero (feature selection)")
print("   - Good when some features may be irrelevant")

# Train Lasso model with optimal parameters from GridSearch
print("\n1. Training Lasso model with optimal parameters...")
lasso = Lasso(
    alpha=lasso_grid.best_params_['alpha'],   # Optimal from GridSearchCV
    random_state=50
)
lasso.fit(X_train_scaled, y_train)
print(f"   Model trained successfully (alpha={lasso_grid.best_params_['alpha']})")

# Make predictions
print("\n2. Making predictions...")
y_pred_lasso = lasso.predict(X_test_scaled)

# Calculate metrics
print("\n3. Calculating performance metrics...")
r2_lasso = r2_score(y_test, y_pred_lasso)
n = len(y_test)
p = len(feature_cols)
adj_r2_lasso = 1 - (1 - r2_lasso) * (n - 1) / (n - p - 1)
rmse_lasso = np.sqrt(mean_squared_error(y_test, y_pred_lasso))
mae_lasso = mean_absolute_error(y_test, y_pred_lasso)

# Cross-validation using consistent kfold
cv_scores_lasso = cross_val_score(lasso, X_train_scaled, y_train,
                                  cv=kfold, scoring='r2')
cv_r2_lasso = cv_scores_lasso.mean()
cv_std_lasso = cv_scores_lasso.std()

# Calculate residuals
```

```
residuals_lasso = y_test - y_pred_lasso

# Display results
print("\n4. Lasso Regression Results:")
print(f"   R2 Score: {r2_lasso:.4f} ({r2_lasso*100:.2f}% variance explained)")
print(f"   Adjusted R2: {adj_r2_lasso:.4f}")
print(f"   RMSE: {rmse_lasso:.2f} percentage points")
print(f"   MAE: {mae_lasso:.2f} percentage points")
print(f"   Cross-Validation R2: {cv_r2_lasso:.4f} (+-{cv_std_lasso:.4f})")

# Feature coefficients
print("\n5. Feature Coefficients:")
coefficients_lasso = pd.DataFrame({
    'Feature': feature_cols,
    'Coefficient': lasso.coef_
}).sort_values('Coefficient', key=abs, ascending=False)
print(coefficients_lasso.to_string(index=False))

# Check for feature selection
zero_coef = (lasso.coef_ == 0).sum()
if zero_coef > 0:
    print(f"\n   Note: Lasso set {zero_coef} coefficient(s) to zero (feature selection)")
else:
    print(f"\n   Note: All features retained (no coefficients set to zero)")

# LASSO VISUALIZATIONS

print("\n6. Creating Lasso visualizations...")

# Create figure with 4 subplots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Lasso Regression - Model Diagnostics', fontsize=16,
fontweight='bold')

# Plot 1: Actual vs Predicted
axes[0, 0].scatter(y_test, y_pred_lasso, alpha=0.6, edgecolors='k', s=50)
axes[0, 0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
                'r--', lw=2, label='Perfect Prediction')
axes[0, 0].set_xlabel('Actual Obesity Prevalence (%)', fontweight='bold')
axes[0, 0].set_ylabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[0, 0].set_title(f'Actual vs Predicted (R² = {r2_lasso:.4f})', fontweight='bold')
axes[0, 0].legend()
axes[0, 0].grid(alpha=0.3)

# Plot 2: Residual Plot
axes[0, 1].scatter(y_pred_lasso, residuals_lasso, alpha=0.6, edgecolors='k',
s=50)
axes[0, 1].axhline(y=0, color='r', linestyle='--', lw=2)
```

```
axes[0, 1].set_xlabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[0, 1].set_ylabel('Residuals', fontweight='bold')
axes[0, 1].set_title('Residual Plot', fontweight='bold')
axes[0, 1].grid(alpha=0.3)

# Plot 3: Residual Distribution
axes[1, 0].hist(residuals_lasso, bins=30, color='steelblue', edgecolor='black',
alpha=0.7)
axes[1, 0].axvline(x=0, color='red', linestyle='--', linewidth=2, label='Mean')
axes[1, 0].set_xlabel('Residuals', fontweight='bold')
axes[1, 0].set_ylabel('Frequency', fontweight='bold')
axes[1, 0].set_title('Distribution of Residuals', fontweight='bold')
axes[1, 0].legend()
axes[1, 0].grid(axis='y', alpha=0.3)

# Plot 4: Feature Coefficients
coef_sorted = coefficients_lasso.sort_values('Coefficient')
colors_coef = ['green' if c > 0 else 'red' for c in coef_sorted['Coefficient']]
axes[1, 1].barh(coef_sorted['Feature'], coef_sorted['Coefficient'],
color=colors_coef, alpha=0.7)
axes[1, 1].axvline(x=0, color='black', linestyle='-', linewidth=1)
axes[1, 1].set_xlabel('Coefficient Value', fontweight='bold')
axes[1, 1].set_title('Feature Coefficients', fontweight='bold')
axes[1, 1].grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Model1_Lasso_Diagnostics.png', dpi=300,
bbox_inches='tight')
plt.show()

print("  ✓ Lasso diagnostics saved: Model1_Lasso_Diagnostics.png")

print("LASSO REGRESSION COMPLETE")

# cross validation
cv_scores_lasso = cross_val_score(lasso, X_train_scaled, y_train, cv=kfold,
scoring='r2')
cv_r2_lasso = cv_scores_lasso.mean()
cv_std_lasso = cv_scores_lasso.std()
```

A.5 Model 2: Ridge Regression

```
# MODEL 2: RIDGE REGRESSION
# Linear regression with L2 regularization (shrinks all coefficients)
# Parameters selected via GridSearchCV (Part 4B)
```

```
print("MODEL 2: RIDGE REGRESSION")

print("\nRidge Regression:")
print("   - Linear regression with L2 regularization")
print("   - Prevents overfitting by penalizing large coefficients")
print("   - Good baseline linear model")

# Train Ridge model with optimal parameters from GridSearch
print("\n1. Training Ridge model with optimal parameters...")
ridge = Ridge(
    alpha=ridge_grid.best_params_['alpha']    # Optimal from GridSearchCV
)
ridge.fit(X_train_scaled, y_train)
print(f"   Model trained successfully (alpha={ridge_grid.best_params_['alpha']})")

# Make predictions
print("\n2. Making predictions...")
y_pred_ridge = ridge.predict(X_test_scaled)

# Calculate metrics
print("\n3. Calculating performance metrics...")
r2_ridge = r2_score(y_test, y_pred_ridge)
adj_r2_ridge = 1 - (1 - r2_ridge) * (n - 1) / (n - p - 1)
rmse_ridge = np.sqrt(mean_squared_error(y_test, y_pred_ridge))
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
cv_scores_ridge = cross_val_score(ridge, X_train_scaled, y_train,
                                  cv=kfold, scoring='r2')
cv_r2_ridge = cv_scores_ridge.mean()
cv_std_ridge = cv_scores_ridge.std()

# Calculate residuals
residuals_ridge = y_test - y_pred_ridge

# Display results
print("\n4. Ridge Regression Results:")
print(f"   R2 Score: {r2_ridge:.4f} ({r2_ridge*100:.2f}% variance explained)")
print(f"   Adjusted R2: {adj_r2_ridge:.4f}")
print(f"   RMSE: {rmse_ridge:.2f} percentage points")
print(f"   MAE: {mae_ridge:.2f} percentage points")
print(f"   Cross-Validation R2: {cv_scores_ridge.mean():.4f} (+-
{cv_scores_ridge.std():.4f})")

# Feature coefficients
print("\n5. Feature Coefficients:")
coefficients_ridge = pd.DataFrame({
    'Feature': feature_cols,
    'Coefficient': ridge.coef_
}).sort_values('Coefficient', key=abs, ascending=False)
```

```
print(coefficients_ridge.to_string(index=False))

# RIDGE VISUALIZATIONS

print("\n6. Creating Ridge visualizations...")

# Create figure with 4 subplots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Ridge Regression - Model Diagnostics', fontsize=16,
fontweight='bold')

# Plot 1: Actual vs Predicted
axes[0, 0].scatter(y_test, y_pred_ridge, alpha=0.6, edgecolors='k', s=50,
color='forestgreen')
axes[0, 0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
                'r--', lw=2, label='Perfect Prediction')
axes[0, 0].set_xlabel('Actual Obesity Prevalence (%)', fontweight='bold')
axes[0, 0].set_ylabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[0, 0].set_title(f'Actual vs Predicted (R² = {r2_ridge:.4f})', fontweight='bold')
axes[0, 0].legend()
axes[0, 0].grid(alpha=0.3)

# Plot 2: Residual Plot
axes[0, 1].scatter(y_pred_ridge, residuals_ridge, alpha=0.6, edgecolors='k', s=50,
color='forestgreen')
axes[0, 1].axhline(y=0, color='r', linestyle='--', lw=2)
axes[0, 1].set_xlabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[0, 1].set_ylabel('Residuals', fontweight='bold')
axes[0, 1].set_title('Residual Plot', fontweight='bold')
axes[0, 1].grid(alpha=0.3)

# Plot 3: Residual Distribution
axes[1, 0].hist(residuals_ridge, bins=30, color='forestgreen', edgecolor='black',
alpha=0.7)
axes[1, 0].axvline(x=0, color='red', linestyle='--', linewidth=2, label='Mean')
axes[1, 0].set_xlabel('Residuals', fontweight='bold')
axes[1, 0].set_ylabel('Frequency', fontweight='bold')
axes[1, 0].set_title('Distribution of Residuals', fontweight='bold')
axes[1, 0].legend()
axes[1, 0].grid(axis='y', alpha=0.3)

# Plot 4: Feature Coefficients
coef_sorted = coefficients_ridge.sort_values('Coefficient')
colors_coef = ['green' if c > 0 else 'red' for c in coef_sorted['Coefficient']]
axes[1, 1].barh(coef_sorted['Feature'], coef_sorted['Coefficient'],
color=colors_coef, alpha=0.7)
axes[1, 1].axvline(x=0, color='black', linestyle='-', linewidth=1)
axes[1, 1].set_xlabel('Coefficient Value', fontweight='bold')
```

```
axes[1, 1].set_title('Feature Coefficients', fontweight='bold')
axes[1, 1].grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Model2_Ridge_Diagnostics.png', dpi=300,
bbox_inches='tight')
plt.show()

print("  ✓ Ridge diagnostics saved: Model2_Ridge_Diagnostics.png")
print("RIDGE REGRESSION COMPLETE")
```

A.6 Model 3: Random Forest

```
# MODEL 3: RANDOM FOREST REGRESSOR
# Ensemble of decision trees (captures non-linear relationships)
# Parameters selected via GridSearchCV (Part 4B)

print("MODEL 3: RANDOM FOREST REGRESSOR")

print("\nRandom Forest:")
print("   - Ensemble of decision trees")
print("   - Captures non-linear relationships")
print("   - Handles feature interactions automatically")

# Train Random Forest with optimal parameters from GridSearch
print("\n1. Training Random Forest model with optimal parameters...")
rf = RandomForestRegressor(
    n_estimators=rf_grid.best_params_['n_estimators'],     # Optimal from
GridSearchCV
    max_depth=rf_grid.best_params_['max_depth'],          # Optimal from
GridSearchCV
    min_samples_split=rf_grid.best_params_['min_samples_split'],  # Optimal
from GridSearchCV
    max_features=rf_grid.best_params_['max_features'],     # Optimal from
GridSearchCV
    random_state=50,
    n_jobs=-1
)
rf.fit(X_train_scaled, y_train)
print(f"   Model trained with {rf.n_estimators} trees")
print(f"   Parameters: {rf_grid.best_params_}")

# Make predictions
print("\n2. Making predictions...")
y_pred_rf = rf.predict(X_test_scaled)
```

```
# Calculate metrics
print("\n3. Calculating performance metrics...")
r2_rf = r2_score(y_test, y_pred_rf)
adj_r2_rf = 1 - (1 - r2_rf) * (n - 1) / (n - p - 1)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
mae_rf = mean_absolute_error(y_test, y_pred_rf)
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train,
                               cv=kfold, scoring='r2')
cv_r2_rf = cv_scores_rf.mean()
cv_std_rf = cv_scores_rf.std()

# Calculate residuals
residuals_rf = y_test - y_pred_rf

# Display results
print("\n4. Random Forest Results:")
print(f"   R2 Score: {r2_rf:.4f} ({r2_rf*100:.2f}% variance explained)")
print(f"   Adjusted R2: {adj_r2_rf:.4f}")
print(f"   RMSE: {rmse_rf:.2f} percentage points")
print(f"   MAE: {mae_rf:.2f} percentage points")
print(f"   Cross-Validation R2: {cv_scores_rf.mean():.4f} (+-
{cv_scores_rf.std():.4f})")

# Feature importance
print("\n5. Feature Importance:")
feature_importance_rf = pd.DataFrame({
    'Feature': feature_cols,
    'Importance': rf.feature_importances_
}).sort_values('Importance', ascending=False)
print(feature_importance_rf.to_string(index=False))

# Behavioral vs socioeconomic contributions
behavioral_importance = feature_importance_rf[
    feature_importance_rf['Feature'].isin(['physical_inactivity',
'smoking_prevalence', 'diet_5_a_day_pct'])
]['Importance'].sum()
socioeconomic_importance = feature_importance_rf[
    feature_importance_rf['Feature'].isin(['imd_score', 'children_low_income_pct'])
]['Importance'].sum()
print(f"\n   Behavioral features contribution: {behavioral_importance*100:.1f}%")
print(f"   Socioeconomic features contribution:
{socioeconomic_importance*100:.1f}%")

# RANDOM FOREST VISUALIZATIONS

print("\n6. Creating Random Forest visualizations...")

# Create figure with 4 subplots
```

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Random Forest - Model Diagnostics', fontsize=16, fontweight='bold')

# Plot 1: Actual vs Predicted
axes[0, 0].scatter(y_test, y_pred_rf, alpha=0.6, edgecolors='k', s=50,
color='coral')
axes[0, 0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
            'r--', lw=2, label='Perfect Prediction')
axes[0, 0].set_xlabel('Actual Obesity Prevalence (%)', fontweight='bold')
axes[0, 0].set_ylabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[0, 0].set_title(f'Actual vs Predicted (R² = {r2_rf:.4f})', fontweight='bold')
axes[0, 0].legend()
axes[0, 0].grid(alpha=0.3)

# Plot 2: Residual Plot
axes[0, 1].scatter(y_pred_rf, residuals_rf, alpha=0.6, edgecolors='k', s=50,
color='coral')
axes[0, 1].axhline(y=0, color='r', linestyle='--', lw=2)
axes[0, 1].set_xlabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[0, 1].set_ylabel('Residuals', fontweight='bold')
axes[0, 1].set_title('Residual Plot', fontweight='bold')
axes[0, 1].grid(alpha=0.3)

# Plot 3: Residual Distribution
axes[1, 0].hist(residuals_rf, bins=30, color='coral', edgecolor='black', alpha=0.7)
axes[1, 0].axvline(x=0, color='red', linestyle='--', linewidth=2, label='Mean')
axes[1, 0].set_xlabel('Residuals', fontweight='bold')
axes[1, 0].set_ylabel('Frequency', fontweight='bold')
axes[1, 0].set_title('Distribution of Residuals', fontweight='bold')
axes[1, 0].legend()
axes[1, 0].grid(axis='y', alpha=0.3)

# Plot 4: Feature Importance
feature_importance_rf_sorted = feature_importance_rf.sort_values('Importance',
ascending=True)
colors_importance = ['#2E86AB' if feat in ['physical_inactivity',
'smoking_prevalence', 'diet_5_a_day_pct']
                 else '#A23B72' for feat in feature_importance_rf_sorted['Feature']]
axes[1, 1].barh(feature_importance_rf_sorted['Feature'],
            feature_importance_rf_sorted['Importance'],
            color=colors_importance, alpha=0.7)
axes[1, 1].set_xlabel('Importance', fontweight='bold')
axes[1, 1].set_title('Feature Importance', fontweight='bold')
axes[1, 1].grid(axis='x', alpha=0.3)

# Add value labels correctly
for i, (idx, row) in enumerate(feature_importance_rf_sorted.iterrows()):
    axes[1, 1].text(row['Importance'] + 0.005, i,
```

```
                f"{row['Importance']:.3f}",
                va='center', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Model3_RandomForest_Diagnostics.png',
dpi=300, bbox_inches='tight')
plt.show()

print("  ✓ Random Forest diagnostics saved:
Model3_RandomForest_Diagnostics.png")

print("RANDOM FOREST COMPLETE")
```

A.7 Model 4: Gradient Boosting

```
# MODEL 4: GRADIENT BOOSTING REGRESSOR
# Sequential ensemble (builds trees to correct previous errors)
# Parameters selected via GridSearchCV (Part 4B)

print("MODEL 4: GRADIENT BOOSTING REGRESSOR")

print("\nGradient Boosting:")
print("   - Builds trees sequentially")
print("   - Each tree corrects errors of previous trees")
print("   - Often achieves high accuracy")

# Train Gradient Boosting with optimal parameters from GridSearch
print("\n1. Training Gradient Boosting model with optimal parameters...")
gb = GradientBoostingRegressor(
    n_estimators=gb_grid.best_params_['n_estimators'],
    max_depth=gb_grid.best_params_['max_depth'],
    learning_rate=gb_grid.best_params_['learning_rate'],
    min_samples_split=gb_grid.best_params_['min_samples_split'],
    subsample=gb_grid.best_params_['subsample'],
    max_features=gb_grid.best_params_['max_features'],
    random_state=50
)
gb.fit(X_train_scaled, y_train)
print(f"   Model trained with {gb.n_estimators} boosting stages")
print(f"   Parameters: {gb_grid.best_params_}")

# Make predictions
print("\n2. Making predictions...")
y_pred_gb = gb.predict(X_test_scaled)

# Calculate metrics
```

```
print("\n3. Calculating performance metrics...")
r2_gb = r2_score(y_test, y_pred_gb)
adj_r2_gb = 1 - (1 - r2_gb) * (n - 1) / (n - p - 1)
rmse_gb = np.sqrt(mean_squared_error(y_test, y_pred_gb))
mae_gb = mean_absolute_error(y_test, y_pred_gb)
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train,
                               cv=kfold, scoring='r2')
cv_r2_gb = cv_scores_gb.mean()
cv_std_gb = cv_scores_gb.std()

# Calculate residuals
residuals_gb = y_test - y_pred_gb

# Display results
print("\n4. Gradient Boosting Results:")
print(f"   R2 Score: {r2_gb:.4f} ({r2_gb*100:.2f}% variance explained)")
print(f"   Adjusted R2: {adj_r2_gb:.4f}")
print(f"   RMSE: {rmse_gb:.2f} percentage points")
print(f"   MAE: {mae_gb:.2f} percentage points")
print(f"   Cross-Validation R2: {cv_scores_gb.mean():.4f} (+-
{cv_scores_gb.std():.4f})")

# Feature importance
print("\n5. Feature Importance:")
feature_importance_gb = pd.DataFrame({
    'Feature': feature_cols,
    'Importance': gb.feature_importances_
}).sort_values('Importance', ascending=False)
print(feature_importance_gb.to_string(index=False))

# GRADIENT BOOSTING VISUALIZATIONS
print("\n6. Creating Gradient Boosting visualizations...")

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Gradient Boosting - Model Diagnostics', fontsize=16,
fontweight='bold')

# Plot 1: Actual vs Predicted
axes[0, 0].scatter(y_test, y_pred_gb, alpha=0.6, edgecolors='k', s=50,
color='purple')
axes[0, 0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
                'r--', lw=2, label='Perfect Prediction')
axes[0, 0].set_xlabel('Actual Obesity Prevalence (%)', fontweight='bold')
axes[0, 0].set_ylabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[0, 0].set_title(f'Actual vs Predicted (R² = {r2_gb:.4f})', fontweight='bold')
axes[0, 0].legend()
axes[0, 0].grid(alpha=0.3)
```

```
# Plot 2: Residual Plot
axes[0, 1].scatter(y_pred_gb, residuals_gb, alpha=0.6, edgecolors='k', s=50,
color='purple')
axes[0, 1].axhline(y=0, color='r', linestyle='--', lw=2)
axes[0, 1].set_xlabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[0, 1].set_ylabel('Residuals', fontweight='bold')
axes[0, 1].set_title('Residual Plot', fontweight='bold')
axes[0, 1].grid(alpha=0.3)

# Plot 3: Residual Distribution
axes[1, 0].hist(residuals_gb, bins=30, color='purple', edgecolor='black',
alpha=0.7)
axes[1, 0].axvline(x=0, color='red', linestyle='--', linewidth=2, label='Mean')
axes[1, 0].set_xlabel('Residuals', fontweight='bold')
axes[1, 0].set_ylabel('Frequency', fontweight='bold')
axes[1, 0].set_title('Distribution of Residuals', fontweight='bold')
axes[1, 0].legend()
axes[1, 0].grid(axis='y', alpha=0.3)

# Plot 4: Feature Importance — GB (fixed)
feature_importance_gb_sorted =
feature_importance_gb.sort_values('Importance', ascending=True)
colors_importance = ['#2E86AB' if feat in ['physical_inactivity',
'smoking_prevalence', 'diet_5_a_day_pct']
                    else '#A23B72' for feat in feature_importance_gb_sorted['Feature']]
axes[1, 1].barh(feature_importance_gb_sorted['Feature'],
               feature_importance_gb_sorted['Importance'],
               color=colors_importance, alpha=0.7)
axes[1, 1].set_xlabel('Importance', fontweight='bold')
axes[1, 1].set_title('Feature Importance', fontweight='bold')
axes[1, 1].grid(axis='x', alpha=0.3)

for i, (idx, row) in enumerate(feature_importance_gb_sorted.iterrows()):
    axes[1, 1].text(row['Importance'] + 0.005, i,
                   f"{row['Importance']:.3f}",
                   va='center', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Model4_GradientBoosting_Diagnostics.png',
dpi=300, bbox_inches='tight')
plt.show()

print("  ✓ Gradient Boosting diagnostics saved:
Model4_GradientBoosting_Diagnostics.png")
print("GRADIENT BOOSTING COMPLETE")

# GRADIENT BOOSTING — SHAPIRO-WILK NORMALITY TEST
# Proving why GB was rejected despite higher CV R²
```

```
from scipy import stats

print("GRADIENT BOOSTING — RESIDUAL NORMALITY TEST")

# Run Shapiro-Wilk test on GB test residuals
stat_gb, p_value_gb = stats.shapiro(residuals_gb)

print(f"\nShapiro-Wilk Test on Test Residuals:")
print(f"   W statistic : {stat_gb:.4f}")
print(f"   p-value     : {p_value_gb:.4f}")

if p_value_gb < 0.05:
    print(f"\n    FAILED — p={p_value_gb:.4f} < 0.05")
    print(f"   Residuals are NOT normally distributed")
    print(f"   This indicates systematic bias in predictions")
else:
    print(f"\n    PASSED — p={p_value_gb:.4f} > 0.05")
    print(f"   Residuals are normally distributed")

print("COMPARISON: RANDOM FOREST vs GRADIENT BOOSTING")

# Run Shapiro-Wilk on RF residuals for direct comparison
stat_rf, p_value_rf = stats.shapiro(residuals_rf)

print(f"\n{'Metric':<35} {'Random Forest':>15} {'Grad. Boosting':>15}")
print(f"{'CV R²':<35} {'0.4388':>15} {'0.5080':>15}")
print(f"{'Test R²':<35} {'0.6012':>15} {'0.6303':>15}")
print(f"{'Train/Test Std Ratio':<35} {'2.5x ':>15} {'10.7x ':>15}")
print(f"{'Shapiro-Wilk W':<35} {stat_rf:>15.4f} {stat_gb:>15.4f}")
print(f"{'Shapiro-Wilk p-value':<35} {p_value_rf:>15.4f} {p_value_gb:>15.4f}")
print(f"{'Normality Test':<35} {'PASS ':>15} {'FAIL ':>15}")
print(f"{'Selected as Final Model':<35} {'YES ':>15} {'NO ':>15}")

print(f"""
CONCLUSION:
Although Gradient Boosting achieved higher CV R² (0.5080 vs 0.4388),
its residuals failed the Shapiro-Wilk normality test (p={p_value_gb:.4f} < 0.05),
indicating systematic bias and significant overfitting (train/test ratio = 10.7x).

Random Forest passed normality (p={p_value_rf:.4f} > 0.05) with an acceptable
train/test ratio of 2.5x, making it the statistically valid final model.

Model selection was based on diagnostic validity, not raw performance metrics.
""")

# GRADIENT BOOSTING — FULL RESIDUAL DIAGNOSTIC PLOTS
# Visual proof of why GB failed normality vs Random Forest
```

```
from scipy import stats
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Residual Analysis — Gradient Boosting Model\n(Predicting Adult
Obesity Prevalence in England)',
           fontsize=15, fontweight='bold')

# ─── PLOT 1: Residuals vs Fitted Values ───
axes[0, 0].scatter(y_pred_gb, residuals_gb, alpha=0.6,
                edgecolors='k', s=50, color='purple')
axes[0, 0].axhline(y=0, color='red', linestyle='--', lw=2)
axes[0, 0].set_xlabel('Fitted Values (Predicted Obesity %)', fontweight='bold')
axes[0, 0].set_ylabel('Residuals', fontweight='bold')
axes[0, 0].set_title('Residuals vs Fitted Values', fontweight='bold')
axes[0, 0].grid(alpha=0.3)

# ─── PLOT 2: Q-Q Plot ───
(osm, osr), (slope, intercept, r) = stats.probplot(residuals_gb, dist="norm")
axes[0, 1].scatter(osm, osr, alpha=0.6, color='purple', edgecolors='k', s=40)
axes[0, 1].plot(osm, slope * np.array(osm) + intercept,
             color='red', lw=2, label='Normal line')
axes[0, 1].set_xlabel('Theoretical Quantiles', fontweight='bold')
axes[0, 1].set_ylabel('Ordered Values', fontweight='bold')
axes[0, 1].set_title('Q-Q Plot (Normality Check)', fontweight='bold')
axes[0, 1].legend()
axes[0, 1].grid(alpha=0.3)

# ─── PLOT 3: Distribution of Residuals ───
axes[1, 0].hist(residuals_gb, bins=30, color='purple',
             edgecolor='black', alpha=0.7)
axes[1, 0].axvline(x=residuals_gb.mean(), color='red', linestyle='--',
                linewidth=2, label=f'Mean = {residuals_gb.mean():.3f}')
axes[1, 0].axvline(x=0, color='green', linestyle='-',
                linewidth=1.5, label='Zero line')
axes[1, 0].set_xlabel('Residual Value', fontweight='bold')
axes[1, 0].set_ylabel('Frequency', fontweight='bold')
axes[1, 0].set_title('Distribution of Residuals', fontweight='bold')
axes[1, 0].legend()
axes[1, 0].grid(axis='y', alpha=0.3)

# Add Shapiro-Wilk result as text on the plot
stat_gb, p_value_gb = stats.shapiro(residuals_gb)
result_text = f'Shapiro-Wilk W = {stat_gb:.4f}\np = {p_value_gb:.4f}   FAILED'
axes[1, 0].text(0.05, 0.92, result_text,
             transform=axes[1, 0].transAxes,
```

```
            fontsize=10, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='#FADBD8', alpha=0.8))

# ─── PLOT 4: Actual vs Predicted ───
axes[1, 1].scatter(y_test, y_pred_gb, alpha=0.6,
                edgecolors='k', s=50, color='purple')
axes[1, 1].plot([y_test.min(), y_test.max()],
            [y_test.min(), y_test.max()],
            'r--', lw=2, label='Perfect Prediction')
axes[1, 1].set_xlabel('Actual Obesity Prevalence (%)', fontweight='bold')
axes[1, 1].set_ylabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[1, 1].set_title('Actual vs Predicted Values', fontweight='bold')
axes[1, 1].legend()
axes[1, 1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/GB_Residual_Analysis.png',
        dpi=300, bbox_inches='tight')
plt.show()

print("✓ GB Residual Analysis saved: GB_Residual_Analysis.png")

# ─── SIDE BY SIDE COMPARISON: RF vs GB Q-Q Plots ───
fig2, axes2 = plt.subplots(1, 2, figsize=(14, 5))
fig2.suptitle('Q-Q Plot Comparison — Random Forest vs Gradient
Boosting\n(Normality of Residuals)',
           fontsize=14, fontweight='bold')

# RF Q-Q
stat_rf, p_value_rf = stats.shapiro(residuals_rf)
(osm_rf, osr_rf), (slope_rf, intercept_rf, r_rf) = stats.probplot(residuals_rf,
dist="norm")
axes2[0].scatter(osm_rf, osr_rf, alpha=0.6, color='coral', edgecolors='k', s=40)
axes2[0].plot(osm_rf, slope_rf * np.array(osm_rf) + intercept_rf, color='red', lw=2)
axes2[0].set_xlabel('Theoretical Quantiles', fontweight='bold')
axes2[0].set_ylabel('Ordered Values', fontweight='bold')
axes2[0].set_title('Random Forest — Q-Q Plot\n PASSES Normality Test',
fontweight='bold', color='green')
axes2[0].grid(alpha=0.3)
axes2[0].text(0.05, 0.95, f'Shapiro-Wilk W = {stat_rf:.4f}\np = {p_value_rf:.4f}
PASS',
           transform=axes2[0].transAxes, fontsize=11, verticalalignment='top',
           bbox=dict(boxstyle='round', facecolor='#D5F5E3', alpha=0.9))

# GB Q-Q
(osm_gb, osr_gb), (slope_gb, intercept_gb, r_gb) = stats.probplot(residuals_gb,
dist="norm")
```

```
axes2[1].scatter(osm_gb, osr_gb, alpha=0.6, color='purple', edgecolors='k',
s=40)
axes2[1].plot(osm_gb, slope_gb * np.array(osm_gb) + intercept_gb, color='red',
lw=2)
axes2[1].set_xlabel('Theoretical Quantiles', fontweight='bold')
axes2[1].set_ylabel('Ordered Values', fontweight='bold')
axes2[1].set_title('Gradient Boosting — Q-Q Plot\n FAILS Normality Test',
fontweight='bold', color='red')
axes2[1].grid(alpha=0.3)
axes2[1].text(0.05, 0.95, f'Shapiro-Wilk W = {stat_gb:.4f}\np = {p_value_gb:.4f}
FAIL',
            transform=axes2[1].transAxes, fontsize=11, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='#FADBD8', alpha=0.9))

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/RF_vs_GB_QQ_Comparison.png',
          dpi=300, bbox_inches='tight')
plt.show()

print("✓ RF vs GB Q-Q Comparison saved: RF_vs_GB_QQ_Comparison.png")
```

A.8 Model Comparison

```
# Part 5: Model Comparison - Data Preparation

print("COMPREHENSIVE MODEL COMPARISON")

# Create comparison dataframe
comparison = pd.DataFrame({
    'Model': ['Lasso Regression', 'Ridge Regression', 'Random Forest', 'Gradient
Boosting'],
    'R2': [r2_lasso, r2_ridge, r2_rf, r2_gb],
    'Adjusted R2': [adj_r2_lasso, adj_r2_ridge, adj_r2_rf, adj_r2_gb],
    'RMSE': [rmse_lasso, rmse_ridge, rmse_rf, rmse_gb],
    'MAE': [mae_lasso, mae_ridge, mae_rf, mae_gb],
    'CV R2': [cv_r2_lasso, cv_r2_ridge, cv_r2_rf, cv_r2_gb],
    'CV Std': [cv_std_lasso, cv_std_ridge, cv_std_rf, cv_std_gb]
})

# Display table
print("\nModel Performance Comparison:")
print(comparison.to_string(index=False))

# Save to CSV
comparison.to_csv('/content/drive/MyDrive/Model_Comparison_Results.csv',
index=False)
```

```
print("\n✓ Comparison table saved: Model_Comparison_Results.csv")

# Part 5: Model Selection Decision

print("MODEL SELECTION DECISION")

# Show all CV R² scores
best_cv_idx = comparison['CV R2'].idxmax()
print(f"\nCross-Validation R² Rankings:")
for idx, row in comparison.iterrows():
    print(f"  {row['Model']:20s}: CV R²={row['CV R2']:.4f}")

# Explicitly select Random Forest based on CV R² AND residual diagnostics
best_model = 'Random Forest'
best_idx = comparison[comparison['Model'] == best_model].index[0]

print(f"\nMODEL SELECTION: {best_model}")
print("\nSelection Rationale:")
print(f"  CV R²: {comparison.loc[best_idx, 'CV R2']:.4f} — strong generalisation")
print(f"  MAE:   {comparison.loc[best_idx, 'MAE']:.4f} — accurate predictions")
print(f"  RMSE:  {comparison.loc[best_idx, 'RMSE']:.4f} — low overall error")
print(f"  Residual normality: Shapiro-Wilk p=0.7566 (PASS)")
print(f"  Training/test residual ratio: 2.5x (minimal overfitting)")
print()
print("NOTE: Gradient Boosting achieved higher CV R² (0.5080 vs 0.4388)")
print("but failed residual normality test (p=0.0431) and showed")
print("significant overfitting (training/test ratio 10.7x).")
print("Random Forest selected based on superior diagnostic quality")
print("and acceptable generalisation performance.")

# Part 5: Model Comparison Visualizations (1/2)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: R² Comparison
x_pos = range(len(comparison))
axes[0].bar(x_pos, comparison['R2'], color='steelblue', alpha=0.8)
axes[0].set_xticks(x_pos)
axes[0].set_xticklabels(comparison['Model'], rotation=45, ha='right')
axes[0].set_ylabel('R² Score', fontweight='bold')
axes[0].set_title('Test R² Comparison', fontweight='bold', fontsize=12)
axes[0].grid(axis='y', alpha=0.3)

# Highlight best
best_r2_idx = comparison['R2'].idxmax()
axes[0].bar(best_r2_idx, comparison.loc[best_r2_idx, 'R2'],
            color='lightcoral', alpha=0.8, label='Highest Test R²')
```

```
axes[0].legend()

# Plot 2: RMSE Comparison
axes[1].bar(x_pos, comparison['RMSE'], color='coral', alpha=0.8)
axes[1].set_xticks(x_pos)
axes[1].set_xticklabels(comparison['Model'], rotation=45, ha='right')
axes[1].set_ylabel('RMSE (percentage points)', fontweight='bold')
axes[1].set_title('RMSE Comparison (Lower is Better)', fontweight='bold',
fontsize=12)
axes[1].grid(axis='y', alpha=0.3)

# Highlight best
best_rmse_idx = comparison['RMSE'].idxmin()
axes[1].bar(best_rmse_idx, comparison.loc[best_rmse_idx, 'RMSE'],
          color='seagreen', alpha=0.8, label='Lowest RMSE')
axes[1].legend()

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Model_Comparison_R2_RMSE.png', dpi=300,
bbox_inches='tight')
plt.show()

# Part 5: Model Comparison Visualizations (2/2)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 3: CV R² Comparison
x_pos = range(len(comparison))
axes[0].bar(x_pos, comparison['CV R2'], color='seagreen', alpha=0.8)
axes[0].errorbar(x_pos, comparison['CV R2'], yerr=comparison['CV Std'],
              fmt='none', ecolor='black', capsize=5, alpha=0.7)
axes[0].set_xticks(x_pos)
axes[0].set_xticklabels(comparison['Model'], rotation=45, ha='right')
axes[0].set_ylabel('CV R²', fontweight='bold')
axes[0].set_title('Cross-Validation R² (Generalization)', fontweight='bold',
fontsize=12)
axes[0].grid(axis='y', alpha=0.3)

# Highlight best (THIS IS THE WINNER!)
best_cv_idx = comparison['CV R2'].idxmax()
axes[0].bar(best_cv_idx, comparison.loc[best_cv_idx, 'CV R2'],
          color='gold', alpha=0.9, label='    Best Generalization')
axes[0].legend()

# Plot 4: Overfitting Gap
gap = comparison['R2'] - comparison['CV R2']
axes[1].bar(x_pos, gap, color='indianred', alpha=0.8)
axes[1].set_xticks(x_pos)
```

```
axes[1].set_xticklabels(comparison['Model'], rotation=45, ha='right')
axes[1].set_ylabel('Gap (Test R² - CV R²)', fontweight='bold')
axes[1].set_title('Generalization Gap (Lower is Better)', fontweight='bold',
fontsize=12)
axes[1].grid(axis='y', alpha=0.3)

# Highlight best (smallest gap)
best_gap_idx = gap.idxmin()
axes[1].bar(best_gap_idx, gap.iloc[best_gap_idx],
          color='lightgreen', alpha=0.8, label='Smallest Gap')
axes[1].legend()

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Model_Comparison_CV_Gap.png', dpi=300,
bbox_inches='tight')
plt.show()

# Part 5: Summary and Key Findings

print("KEY FINDINGS FROM MODEL COMPARISON")

print("\n1. FINAL MODEL: Random Forest")
print(f"   - CV R²: {comparison.loc[best_idx, 'CV R2']:.4f}")
print(f"   - MAE:   {comparison.loc[best_idx, 'MAE']:.2f}")
print(f"   - RMSE:  {comparison.loc[best_idx, 'RMSE']:.2f}")
print(f"   - Residual normality: p=0.7566 (PASS)")
print(f"   - Training/test ratio: 2.5x (minimal overfitting)")

print("\n2. MODEL PROGRESSION:")
for idx, row in comparison.iterrows():
    print(f"   {row['Model']:20s}: R²={row['R2']:.4f}, CV R²={row['CV R2']:.4f}")

print("\n3. ENSEMBLE METHODS SUPERIOR:")
print("   Linear models (Lasso/Ridge): CV R² = 0.28")
print("   Ensemble methods (RF/GB):    CV R² = 0.44-0.51")
print("   Improvement: approximately 57-82% better generalisation")

print("\n4. WHY RANDOM FOREST OVER GRADIENT BOOSTING:")
print(f"   - RF residual normality: p=0.7566 vs GB: p=0.0431")
print(f"   - RF training/test ratio: 2.5x vs GB: 10.7x")
print(f"   - GB higher CV R² (0.5080) but fails diagnostic criteria")
print(f"   - RF provides reliable, well-fitted model for this dataset")
```

A.9 SHAP Feature Importance Analysis

```
import matplotlib.patches as mpatches
import matplotlib.pyplot as plt
```

```
import pandas as pd
import numpy as np

# PART 6: FEATURE IMPORTANCE ANALYSIS (Random Forest)

print("PART 6: FEATURE IMPORTANCE ANALYSIS")

# Get feature importance from Random Forest (Final Model)
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf.feature_importances_
}).sort_values('Importance', ascending=False)

print("\nFeature Importance Rankings:")
print(feature_importance.to_string(index=False))

fig, ax = plt.subplots(figsize=(10, 6))

colours = ['#2ecc71' if i == 0 else '#3498db' if i <= 2 else '#95a5a6'
           for i in range(len(feature_importance))]

bars = ax.barh(feature_importance['Feature'],
               feature_importance['Importance'],
               color=colours, edgecolor='white', linewidth=0.5)

ax.set_xlabel('Feature Importance Score', fontweight='bold')
ax.set_title('Random Forest — Feature Importance\n(Predicting Adult Obesity
Prevalence in England)',
             fontweight='bold', fontsize=13)
ax.invert_yaxis()

for bar, val in zip(bars, feature_importance['Importance']):
    ax.text(bar.get_width() + 0.001, bar.get_y() + bar.get_height()/2,
            f'{val:.4f}', va='center', fontsize=10)

top1 = mpatches.Patch(color='#2ecc71', label='Top Predictor')
top3 = mpatches.Patch(color='#3498db', label='2nd-3rd Predictor')
rest = mpatches.Patch(color='#95a5a6', label='Other Predictors')
ax.legend(handles=[top1, top3, rest], loc='lower right')

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Feature_Importance.png', dpi=300,
bbox_inches='tight')
plt.show()

print("\nFeature importance plot saved!")

# PART 6B: SHAP ANALYSIS (Random Forest — Final Model)
```

```
try:
    import shap
except ImportError:
    import subprocess
    subprocess.run(['pip', 'install', 'shap', '-q'])
    import shap

print("PART 7: SHAP ANALYSIS (Random Forest)")

explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X_test_scaled)

print(f"\nSHAP values computed for {X_test_scaled.shape[0]} test observations")
print(f"Features analysed: {X_test_scaled.shape[1]}")

print("\nGenerating SHAP Summary Plot...")
plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values, X_test_scaled,
                  feature_names=X.columns.tolist(),
                  show=False)
plt.title('SHAP Summary Plot — Impact on Obesity Prevalence Prediction',
          fontweight='bold', fontsize=13, pad=15)
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/SHAP_Summary.png', dpi=300,
bbox_inches='tight')
plt.show()
print("SHAP Summary plot saved!")

print("\nGenerating SHAP Bar Chart...")
plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values, X_test_scaled,
                  feature_names=X.columns.tolist(),
                  plot_type='bar',
                  show=False)
plt.title('SHAP Feature Importance — Mean Absolute SHAP Values',
          fontweight='bold', fontsize=13, pad=15)
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/SHAP_Bar.png', dpi=300,
bbox_inches='tight')
plt.show()
print("SHAP Bar chart saved!")

print("SHAP SUMMARY STATISTICS")

mean_shap = pd.DataFrame({
    'Feature': X.columns,
    'Mean |SHAP|': np.abs(shap_values).mean(axis=0)
```

```
}).sort_values('Mean |SHAP|', ascending=False)

print("\nFeature ranking by mean absolute SHAP value:")
print(mean_shap.to_string(index=False))

print("\nInterpretation:")
print(f"  Top predictor: {mean_shap.iloc[0]['Feature']} "
      f"(SHAP = {mean_shap.iloc[0]['Mean |SHAP|']:.3f})")
print(f"  2nd predictor: {mean_shap.iloc[1]['Feature']} "
      f"(SHAP = {mean_shap.iloc[1]['Mean |SHAP|']:.3f})")
print(f"  3rd predictor: {mean_shap.iloc[2]['Feature']} "
      f"(SHAP = {mean_shap.iloc[2]['Mean |SHAP|']:.3f})")


# PART 6C: SHAP DEPENDENCE PLOTS (Top 2 Predictors)

top_features = mean_shap['Feature'].iloc[:2].tolist()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for i, feature in enumerate(top_features):
    feat_idx = list(X.columns).index(feature)
    axes[i].scatter(X_test_scaled[:, feat_idx],
                    shap_values[:, feat_idx],
                    alpha=0.6, c=shap_values[:, feat_idx],
                    cmap='coolwarm', edgecolors='none')
    axes[i].axhline(y=0, color='black', linestyle='--', linewidth=1)
    axes[i].set_xlabel(feature, fontweight='bold')
    axes[i].set_ylabel('SHAP Value', fontweight='bold')
    axes[i].set_title(f'SHAP Dependence: {feature}', fontweight='bold')

plt.suptitle('SHAP Dependence Plots — How Top Predictors Affect Obesity
Prevalence',
             fontweight='bold', fontsize=13, y=1.02)
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/SHAP_Dependence.png', dpi=300,
bbox_inches='tight')
plt.show()
print("SHAP Dependence plots saved!")
```

A.10 Residual Analysis and Model Diagnostics

```
import scipy.stats as stats

print("PART 8: RESIDUAL ANALYSIS & MODEL DIAGNOSTICS")

# Generate predictions using Random Forest (Final Model)
```

```
y_pred_train = rf.predict(X_train_scaled)
y_pred_test = rf.predict(X_test_scaled)

residuals_train = y_train - y_pred_train
residuals_test = y_test - y_pred_test

print(f"\nTraining Set Residuals:")
print(f"  Mean:     {residuals_train.mean():.4f} (should be close to 0)")
print(f"  Std Dev:  {residuals_train.std():.4f}")
print(f"  Min:      {residuals_train.min():.4f}")
print(f"  Max:      {residuals_train.max():.4f}")

print(f"\nTest Set Residuals:")
print(f"  Mean:     {residuals_test.mean():.4f} (should be close to 0)")
print(f"  Std Dev:  {residuals_test.std():.4f}")
print(f"  Min:      {residuals_test.min():.4f}")
print(f"  Max:      {residuals_test.max():.4f}")

stat, p_value = stats.shapiro(residuals_test)
print(f"\nShapiro-Wilk Normality Test (Test Residuals):")
print(f"  Statistic: {stat:.4f}")
print(f"  P-value:   {p_value:.4f}")
print(f"  Result:    {'Residuals are normally distributed' if p_value > 0.05 else
'Residuals deviate from normality'}")

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Residual Analysis — Random Forest Model\n(Predicting Adult
Obesity Prevalence in England)',
           fontweight='bold', fontsize=14, y=1.01)

axes[0, 0].scatter(y_pred_test, residuals_test,
                alpha=0.6, color='#3498db', edgecolors='none')
axes[0, 0].axhline(y=0, color='red', linestyle='--', linewidth=1.5)
axes[0, 0].set_xlabel('Fitted Values (Predicted Obesity %)', fontweight='bold')
axes[0, 0].set_ylabel('Residuals', fontweight='bold')
axes[0, 0].set_title('Residuals vs Fitted Values', fontweight='bold')
axes[0, 0].grid(axis='y', alpha=0.3)

stats.probplot(residuals_test, dist="norm", plot=axes[0, 1])
axes[0, 1].set_title('Q-Q Plot (Normality Check)', fontweight='bold')
axes[0, 1].get_lines()[0].set(color='#3498db', alpha=0.6)
axes[0, 1].get_lines()[1].set(color='red', linewidth=1.5)

axes[1, 0].hist(residuals_test, bins=20,
             color='#3498db', edgecolor='white',
             linewidth=0.5, alpha=0.8)
axes[1, 0].axvline(x=0, color='red', linestyle='--', linewidth=1.5)
axes[1, 0].axvline(x=residuals_test.mean(), color='green',
```

```
                linestyle='-', linewidth=1.5,
                label=f'Mean = {residuals_test.mean():.3f}')
axes[1, 0].set_xlabel('Residual Value', fontweight='bold')
axes[1, 0].set_ylabel('Frequency', fontweight='bold')
axes[1, 0].set_title('Distribution of Residuals', fontweight='bold')
axes[1, 0].legend()
axes[1, 0].grid(axis='y', alpha=0.3)

axes[1, 1].scatter(y_test, y_pred_test,
                alpha=0.6, color='#2ecc71', edgecolors='none')
min_val = min(y_test.min(), y_pred_test.min())
max_val = max(y_test.max(), y_pred_test.max())
axes[1, 1].plot([min_val, max_val], [min_val, max_val],
              'r--', linewidth=1.5, label='Perfect Prediction')
axes[1, 1].set_xlabel('Actual Obesity Prevalence (%)', fontweight='bold')
axes[1, 1].set_ylabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[1, 1].set_title('Actual vs Predicted Values', fontweight='bold')
axes[1, 1].legend()
axes[1, 1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Residual_Analysis.png', dpi=300,
bbox_inches='tight')
plt.show()

print("\nResidual analysis plots saved!")
```

A.11 Geographic Visualisation

```
print("PART 8: GEOGRAPHIC VISUALISATION")

# Install required libraries
try:
    import geopandas as gpd
except ImportError:
    import subprocess
    subprocess.run(['pip', 'install', 'geopandas', '-q'])
    import geopandas as gpd

# Create results dataframe
results_df = pd.DataFrame({
    'actual': y_test.values,
    'predicted': y_pred_test,
    'residual': residuals_test
}, index=y_test.index)

print(f"\nResults dataframe shape: {results_df.shape}")
```

```
print(f"\nSample of results:")
print(results_df.head(10).round(2))

# PLOT: Actual vs Predicted vs Residual Comparison

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle('Geographic Distribution — Adult Obesity Prevalence
Predictions\n(English Local Authorities)',
           fontweight='bold', fontsize=14)

# Sort by actual for cleaner visualisation
results_sorted = results_df.sort_values('actual').reset_index()

x = range(len(results_sorted))

# --- Panel 1: Actual vs Predicted scatter ---
axes[0].scatter(results_sorted['actual'], results_sorted['predicted'],
              alpha=0.6, color='#3498db', edgecolors='none', s=40)
min_val = min(results_sorted['actual'].min(), results_sorted['predicted'].min())
max_val = max(results_sorted['actual'].max(), results_sorted['predicted'].max())
axes[0].plot([min_val, max_val], [min_val, max_val],
           'r--', linewidth=1.5, label='Perfect Prediction')
axes[0].set_xlabel('Actual Obesity Prevalence (%)', fontweight='bold')
axes[0].set_ylabel('Predicted Obesity Prevalence (%)', fontweight='bold')
axes[0].set_title('Actual vs Predicted', fontweight='bold')
axes[0].legend()
axes[0].grid(alpha=0.3)

# --- Panel 2: Ranked actual vs predicted ---
axes[1].plot(x, results_sorted['actual'],
           color='#e74c3c', linewidth=1.5, label='Actual', alpha=0.8)
axes[1].plot(x, results_sorted['predicted'],
           color='#3498db', linewidth=1.5, label='Predicted',
           linestyle='--', alpha=0.8)
axes[1].fill_between(x, results_sorted['actual'], results_sorted['predicted'],
                  alpha=0.2, color='#95a5a6')
axes[1].set_xlabel('Local Authorities (ranked by actual)', fontweight='bold')
axes[1].set_ylabel('Obesity Prevalence (%)', fontweight='bold')
axes[1].set_title('Ranked Prediction Comparison', fontweight='bold')
axes[1].legend()
axes[1].grid(alpha=0.3)

# --- Panel 3: Residuals ranked ---
colours = ['#e74c3c' if r < 0 else '#2ecc71' for r in results_sorted['residual']]
axes[2].bar(x, results_sorted['residual'], color=colours, alpha=0.7, width=1.0)
axes[2].axhline(y=0, color='black', linewidth=1.5)
axes[2].axhline(y=results_sorted['residual'].mean(),
              color='blue', linestyle='--', linewidth=1.5,
```

```
                label=f'Mean = {results_sorted["residual"].mean():.2f}')
axes[2].set_xlabel('Local Authorities (ranked by actual)', fontweight='bold')
axes[2].set_ylabel('Residual (Actual - Predicted)', fontweight='bold')
axes[2].set_title('Residuals by Local Authority', fontweight='bold')
axes[2].legend()
axes[2].grid(axis='y', alpha=0.3)

# Legend for residuals
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor='#e74c3c', alpha=0.7, label='Over-
predicted'),
                   Patch(facecolor='#2ecc71', alpha=0.7, label='Under-predicted')]
axes[2].legend(handles=legend_elements)

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Geographic_Visualisation.png', dpi=300,
bbox_inches='tight')
plt.show()

print("\nGeographic visualisation saved!")

# Summary stats
print("\nPrediction Accuracy Summary:")
print(f"  Local Authorities within ±2%:  "
      f"{(np.abs(residuals_test) <= 2).sum()} / {len(residuals_test)} "
      f"({(np.abs(residuals_test) <= 2).mean()*100:.1f}%)")
print(f"  Local Authorities within ±5%:  "
      f"{(np.abs(residuals_test) <= 5).sum()} / {len(residuals_test)} "
      f"({(np.abs(residuals_test) <= 5).mean()*100:.1f}%)")
print(f"  Local Authorities within ±10%: "
      f"{(np.abs(residuals_test) <= 10).sum()} / {len(residuals_test)} "
      f"({(np.abs(residuals_test) <= 10).mean()*100:.1f}%)")
```

A.12 Model Limitations

A.13 Conclusions and Summary

```
# FINAL COMPLETE OUTPUT — ALL MODELS

print("=== LASSO ===")
print(f"R2: {r2_lasso:.4f}, AdjR2: {adj_r2_lasso:.4f}, RMSE: {rmse_lasso:.4f}, MAE:
{mae_lasso:.4f}, CVR2: {cv_r2_lasso:.4f}, CVStd: {cv_std_lasso:.4f}")
print("Coefficients:")
print(coefficients_lasso.to_string(index=False))
```

```
print("\n=== RIDGE ===")
print(f"R2: {r2_ridge:.4f}, AdjR2: {adj_r2_ridge:.4f}, RMSE: {rmse_ridge:.4f}, MAE:
{mae_ridge:.4f}, CVR2: {cv_r2_ridge:.4f}, CVStd: {cv_std_ridge:.4f}")
print("Coefficients:")
print(coefficients_ridge.to_string(index=False))

print("\n=== RANDOM FOREST ===")
print(f"R2: {r2_rf:.4f}, AdjR2: {adj_r2_rf:.4f}, RMSE: {rmse_rf:.4f}, MAE:
{mae_rf:.4f}, CVR2: {cv_r2_rf:.4f}, CVStd: {cv_std_rf:.4f}")
print("Feature Importance:")
print(feature_importance_rf.to_string(index=False))

print("\n=== GRADIENT BOOSTING ===")
print(f"R2: {r2_gb:.4f}, AdjR2: {adj_r2_gb:.4f}, RMSE: {rmse_gb:.4f}, MAE:
{mae_gb:.4f}, CVR2: {cv_r2_gb:.4f}, CVStd: {cv_std_gb:.4f}")
print("Feature Importance:")
print(feature_importance_gb.to_string(index=False))

print("\n=== MODEL COMPARISON ===")
print(comparison.to_string(index=False))

print("\n=== BEST MODEL ===")
print(f"Selected: {best_model}")

print("\n=== GRIDSEARCH BEST PARAMS ===")
print(f"Lasso alpha:  {lasso_grid.best_params_['alpha']}")
print(f"Ridge alpha:  {ridge_grid.best_params_['alpha']}")
print(f"RF params:    {rf_grid.best_params_}")
print(f"GB params:    {gb_grid.best_params_}")

print("\n=== FEATURE IMPORTANCE (Part 6) ===")
print(feature_importance.to_string(index=False))

print("\n=== SHAP ===")
print(mean_shap.to_string(index=False))

print("\n=== RESIDUALS ===")
print(f"Train Mean: {residuals_train.mean():.4f}, Std: {residuals_train.std():.4f}, Min:
{residuals_train.min():.4f}, Max: {residuals_train.max():.4f}")
print(f"Test Mean:  {residuals_test.mean():.4f}, Std: {residuals_test.std():.4f}, Min:
{residuals_test.min():.4f}, Max: {residuals_test.max():.4f}")
print(f"Shapiro W: {stat:.4f}, p: {p_value:.4f}")

print("\n=== PREDICTION ACCURACY ===")
print(f"Within 2%:  {(np.abs(residuals_test) <= 2).mean()*100:.1f}%")
print(f"Within 5%:  {(np.abs(residuals_test) <= 5).mean()*100:.1f}%")
print(f"Within 10%: {(np.abs(residuals_test) <= 10).mean()*100:.1f}%")
```